



AMBA[®] LTI

Protocol Specification

Document number	ARM IHI 0089
Document quality	Released
Document version	Issue D
Document confidentiality	Non-confidential
Date of issue	13 Aug 2025

Copyright © 2020, 2021, 2023, 2024, 2025 Arm Limited or its affiliates. All rights reserved.

AMBA® LTI Protocol Specification

Release information

Date	Version	Changes
2025/Aug/13	D	• EAC release of Issue D.
2024/May/23	C	• Release with support for Device Permission Table (DPT) and coherent interfaces
2023/Sep/26	B	• Release with support for Realm Management Extension (RME) and Memory Encryption Contexts (MEC)
2021/Jul/21	A.b	• Terminology update
2020/Apr/07	A	• First release

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited (“Arm”). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm’s view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED “AS IS”. ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party’s products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <http://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2020, 2021, 2023, 2024, 2025 Arm Limited or its affiliates. All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-21451 version 3

AMBA SPECIFICATION LICENCE

THIS END USER LICENCE AGREEMENT (“LICENCE”) IS A LEGAL AGREEMENT BETWEEN YOU (EITHER A SINGLE INDIVIDUAL, OR SINGLE LEGAL ENTITY) AND ARM LIMITED (“ARM”) FOR THE USE OF ARM’S INTELLECTUAL PROPERTY (INCLUDING, WITHOUT LIMITATION, ANY COPYRIGHT) IN THE RELEVANT AMBA SPECIFICATION ACCOMPANYING THIS LICENCE. ARM LICENSES THE RELEVANT AMBA SPECIFICATION TO YOU ON CONDITION THAT YOU ACCEPT ALL OF THE TERMS IN THIS LICENCE. BY CLICKING “I AGREE” OR OTHERWISE USING OR COPYING THE RELEVANT AMBA SPECIFICATION YOU INDICATE THAT YOU AGREE TO BE BOUND BY ALL THE TERMS OF THIS LICENCE.

“LICENSEE” means You and your Subsidiaries. “Subsidiary” means, if You are a single entity, any company the majority of whose voting shares is now or hereafter owned or controlled, directly or indirectly, by You. A company shall be a Subsidiary only for the period during which such control exists.

1. Subject to the provisions of Clauses 2, 3 and 4, Arm hereby grants to LICENSEE a perpetual, non-exclusive, non-transferable, royalty free, worldwide licence to:
 - (i) use and copy the relevant AMBA Specification for the purpose of developing and having developed products that comply with the relevant AMBA Specification;
 - (ii) manufacture and have manufactured products which either: (a) have been created by or for LICENSEE under the licence granted in Clause 1(i); or (b) incorporate a product(s) which has been created by a third party(s) under a licence granted by Arm in Clause 1(i) of such third party’s AMBA Specification Licence; and
 - (iii) offer to sell, sell, supply or otherwise distribute products which have either been (a) created by or for LICENSEE under the licence granted in Clause 1(i); or (b) manufactured by or for LICENSEE under the licence granted in Clause 1(ii).
2. LICENSEE hereby agrees that the licence granted in Clause 1 is subject to the following restrictions:
 - (i) where a product created under Clause 1(i) is an integrated circuit which includes a CPU then either: (a) such CPU shall only be manufactured under licence from Arm; or (b) such CPU is neither substantially compliant with nor marketed as being compliant with the Arm instruction sets licensed by Arm from time to time;
 - (ii) the licences granted in Clause 1(iii) shall not extend to any portion or function of a product that is not itself compliant with part of the relevant AMBA Specification; and
 - (iii) no right is granted to LICENSEE to sublicense the rights granted to LICENSEE under this Agreement.
3. Except as specifically licensed in accordance with Clause 1, LICENSEE acquires no right, title or interest in any Arm technology or any intellectual property embodied therein. In no event shall the licences granted in accordance with Clause 1 be construed as granting LICENSEE, expressly or by implication, estoppel or otherwise, a licence to use any Arm technology except the relevant AMBA Specification.
4. THE RELEVANT AMBA SPECIFICATION IS PROVIDED “AS IS” WITH NO REPRESENTATION OR WARRANTIES EXPRESS, IMPLIED OR STATUTORY, INCLUDING BUT NOT LIMITED TO ANY WARRANTY OF SATISFACTORY QUALITY, MERCHANTABILITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE, OR THAT ANY USE OR IMPLEMENTATION OF SUCH ARM TECHNOLOGY WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADE SECRETS OR OTHER INTELLECTUAL PROPERTY RIGHTS.
5. NOTWITHSTANDING ANYTHING TO THE CONTRARY CONTAINED IN THIS AGREEMENT, TO THE FULLEST EXTENT PERMITTED BY LAW, THE MAXIMUM LIABILITY OF ARM IN AGGREGATE FOR ALL CLAIMS MADE AGAINST ARM, IN CONTRACT, TORT OR OTHERWISE, IN CONNECTION WITH THE SUBJECT MATTER OF THIS AGREEMENT (INCLUDING WITHOUT LIMITATION (I) LICENSEE’S USE OF THE ARM TECHNOLOGY; AND (II) THE IMPLEMENTATION OF THE ARM TECHNOLOGY IN ANY PRODUCT CREATED BY LICENSEE UNDER THIS AGREEMENT) SHALL NOT EXCEED THE FEES PAID (IF ANY) BY LICENSEE TO ARM UNDER THIS AGREEMENT. THE EXISTENCE OF MORE THAN ONE CLAIM OR SUIT WILL NOT ENLARGE OR EXTEND THE LIMIT. LICENSEE RELEASES ARM FROM ALL OBLIGATIONS, LIABILITY, CLAIMS OR DEMANDS IN EXCESS OF THIS LIMITATION.

6. No licence, express, implied or otherwise, is granted to LICENSEE, under the provisions of Clause 1, to use the Arm tradename, or AMBA trademark in connection with the relevant AMBA Specification or any products based thereon. Nothing in Clause 1 shall be construed as authority for LICENSEE to make any representations on behalf of Arm in respect of the relevant AMBA Specification.
7. This Licence shall remain in force until terminated by you or by Arm. Without prejudice to any of its other rights if LICENSEE is in breach of any of the terms and conditions of this Licence then Arm may terminate this Licence immediately upon giving written notice to You. You may terminate this Licence at any time. Upon expiry or termination of this Licence by You or by Arm LICENSEE shall stop using the relevant AMBA Specification and destroy all copies of the relevant AMBA Specification in your possession together with all documentation and related materials. Upon expiry or termination of this Licence, the provisions of clauses 6 and 7 shall survive.
8. The validity, construction and performance of this Agreement shall be governed by English Law.

PRE-21451 version 3

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

AMBA® LTI Protocol Specification

AMBA® LTI Protocol Specification	ii
Release information	ii
Non-Confidential Proprietary Notice	iii
AMBA SPECIFICATION LICENCE	iv
Confidentiality Status	v
Product Status	v
Web Address	v

Part A Preface

About this specification

Intended audience	xi
-----------------------------	----

Using this specification

Conventions	xiv
Typographical conventions	xiv
Timing diagrams	xiv
Signals	xiv
Numbers	xv
Additional reading	xvi
Arm publications	xvi
Other publications	xvi
Feedback	xvii
Feedback on this specification	xvii
Inclusive terminology commitment	xvii

Part B Specification

Chapter B1

Introduction

B1.1	About the LTI protocol	20
B1.2	Use cases	21
B1.2.1	In-line integration	21
B1.2.2	Lookaside integration	21
B1.2.3	Cached integration	22
B1.3	Differences between DTI and LTI	23
B1.4	Supported translation flows	24
B1.4.1	Stall flow	24
B1.4.2	ATST flow	24
B1.4.3	NoStall flow	24
B1.4.4	PRI flow	25
B1.5	Translation cache	26

Chapter B2

Channels

B2.1	Transaction flow	28
B2.2	Virtual Channels	30
B2.3	Flow control	31
B2.4	Reserved encodings	32

	B2.5	Signal validity	33
Chapter B3		Properties	
	B3.1	LTI properties	35
Chapter B4		Request channel	
	B4.1	Signals	39
	B4.2	Transaction types	47
	B4.3	Transaction attributes	48
	B4.4	Transaction scope	50
	B4.5	Context and PAS combinations	51
Chapter B5		Response channel	
	B5.1	Signals	53
	B5.2	LRRESP details	59
	B5.2.1	Restrictions based on LATRANS	59
	B5.2.2	Downgrade types	59
	B5.2.3	Restrictions based on LAFLOW	60
	B5.2.4	Out of range address	60
	B5.3	Attribute restrictions for specific transaction types	61
Chapter B6		Completion channel	
	B6.1	Signals	63
	B6.2	Completion channel characteristics	64
	B6.2.1	Deadlock avoidance	64
	B6.2.2	Use of LTI translations for multiple transactions	64
Chapter B7		Tag channel	
	B7.1	Signals	67
	B7.2	Tag message	68
	B7.2.1	Example invalidation sequences	68
	B7.3	Race between LT and LR channels	71
Chapter B8		Interface management	
	B8.1	Interface management overview	73
	B8.2	Open and close handshake	74
	B8.3	Properties of interface states	75
	B8.4	Management signals	76
	B8.4.1	LMASKCLOSE	76
	B8.4.2	LMACTIVE	76
Chapter B9		Clock and reset	
	B9.1	Clock and reset	80
Chapter B10		Pipelining	
	B10.1	Pipelining between Manager and Subordinate interfaces	82
Chapter B11		Interface protection	
	B11.1	Parity protection for interface signals	84
	B11.1.1	Error detection behavior	84
	B11.1.2	Parity check signals	84
Chapter B12		Interoperability	
	B12.1	LTI-A compatibility	89
	B12.2	LTI_MMU and LTI_GPC properties	90
	B12.3	MPAM compatibility	91

B12.4	LAMECID and LAHWATTR signals compatibility	92
B12.5	LTI_CACHE_SUPPORT compatibility	93
B12.6	LTI_GDI compatibility	94
B12.7	LTI_PASUNKNOWN_PRESENT commpatibility	95
B12.8	LTI_OUTER_CACHEABLE_MODE compatibiltiy	96
B12.9	PAS signaling compatibility	97
B12.10	PRIV and INST signaling compatibility	98

Chapter B13

Pseudocode

B13.1	PermissionCheckDHCMO	100
-------	--------------------------------	-----

Part C Appendices

Chapter C1

Considerations for AXI5

C1.1	LATRANS mapping	103
C1.2	LAATTR mapping	105
C1.3	LAIDENT mapping	106
C1.4	LRATTR mapping	107
C1.5	LRMECID mapping	108
C1.6	LRHWATTR mapping	109
C1.7	xRESP mapping	110
C1.8	PAS mapping	111
C1.9	PRIV and INST mapping	112
C1.10	Transactions that are legal in AXI5 and illegal in LTI	113
C1.11	Memory tagging	114
C1.12	AXI opcode modifications	115

Chapter C2

Considerations for DTI

C2.1	DTI_TBU_TRANS_REQ.PERM mapping	117
C2.2	Special handling for specific LATRANS values	118
C2.2.1	LATRANS = DCP	118
C2.2.2	LATRANS = W-DCP	118
C2.2.3	LATRANS = CMO	118
C2.2.4	LATRANS = R-CMO	118
C2.2.5	LATRANS = DCMO	118
C2.2.6	LATRANS = R-DCMO	118
C2.2.7	LATRANS = DHCMO	119
C2.3	Attribute mapping	120
C2.3.1	LTI to Armv8 conversion	120
C2.3.2	Armv8 to LTI conversion	121
C2.4	DTI_TBU_TRANS_FAULT.FAULT_TYPE mapping	124

Chapter C3

Considerations for PCIe

C3.1	PCIe integration	126
------	----------------------------	-----

Chapter C4

Considerations for coherent interfaces

C4.1	LTI properties	130
C4.2	LA mapping from CHI	131
C4.3	LATRANS mapping	132
C4.4	LAATTR mapping	134
C4.5	LRATTR mapping	135
C4.6	LRMECID mapping	137
C4.7	LRHWATTR mapping	138
C4.8	CHI opcode considerations	139
C4.9	CHI completion response flit opcode mapping	140

	C4.10	Transaction flow examples	141
	C4.10.1	Coherent read from CHI C2C device	141
	C4.10.2	Coherent write from CXL.cache device	142
Chapter C5	Signal list		
	C5.1	Signal list	146
Chapter C6	Parity check signal matrix		
Chapter C7	Revisions		
	C7.1	Changes between Issue A and Issue A.b	153
	C7.2	Changes between Issue A.b and Issue B	154
	C7.3	Changes between Issue B and Issue C	155
	C7.4	Changes between Issue C and Issue D	156

Part A

Preface

About this specification

This specification describes the AMBA[®] LTI protocol.

Intended audience

This specification is written for hardware engineers who wish to design components that implement *Local Translation Interface* (LTI).

Using this specification

The information in this specification is organized into parts, as described in this section:

[Chapter B1 *Introduction*](#)

Introduction to the AMBA LTI protocol specification.

[Chapter B2 *Channels*](#)

Describes LTI information flow.

[Chapter B3 *Properties*](#)

Describes the set of LTI properties that specify the supported behavior and interface signaling requirements.

[Chapter B4 *Request channel*](#)

Defines the LTI request (LA) channel.

[Chapter B5 *Response channel*](#)

Defines the LTI response (LR) channel.

[Chapter B6 *Completion channel*](#)

Defines the LTI completion (LC) channel.

[Chapter B7 *Tag channel*](#)

Defines the LTI tag (LT) channel.

[Chapter B8 *Interface management*](#)

Describes the LTI interface management function.

[Chapter B9 *Clock and reset*](#)

Describes a mapping strategy for clock and reset.

[Chapter B10 *Pipelining*](#)

Defines pipelining requirements for LTI.

[Chapter B11 *Interface protection*](#)

Defines the interface protection schemes using parity signaling.

[Chapter B12 *Interoperability*](#)

Describes how to connect LTI interfaces with different properties.

[Chapter B13 *Pseudocode*](#)

Describes the pseudocode used in LTI.

[Chapter C1 *Considerations for AXI5*](#)

Describes how to map LTI concepts onto AXI5.

[Chapter C2 *Considerations for DTI*](#)

Describes how to map LTI concepts onto DTI-TBU.

[Chapter C3 *Considerations for PCIe*](#)

Describes the integration of PCIe.

Chapter C4 Considerations for coherent interfaces

Describes how to map LTI concepts onto coherent interfaces.

Chapter C5 Signal list

Describes which signals are present on the LTI interfaces.

Chapter C6 Parity check signal matrix

Describes which parity signals are present on the LTI interfaces.

Chapter C7 Revisions

Describes the technical changes between issues of this specification.

Conventions

Typographical conventions

The typographical conventions are:

<i>italic</i>	Highlights important notes, introduces special terminology, and denotes internal cross-references and citations.
bold	Denotes signal names, and is used for terms in descriptive lists, where appropriate.
monospace	Used for assembler syntax descriptions, pseudocode, and source code examples. Also used in the main text for instruction mnemonics and for references to other items appearing in assembler syntax descriptions, pseudocode, and source code examples.
SMALL CAPITALS	Used for a few terms that have specific technical meanings.

Timing diagrams

The components used in timing diagrams are explained in [Figure 1](#). Variations have clear labels, when they occur. Do not assume any timing information that is not explicit in the diagrams.

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

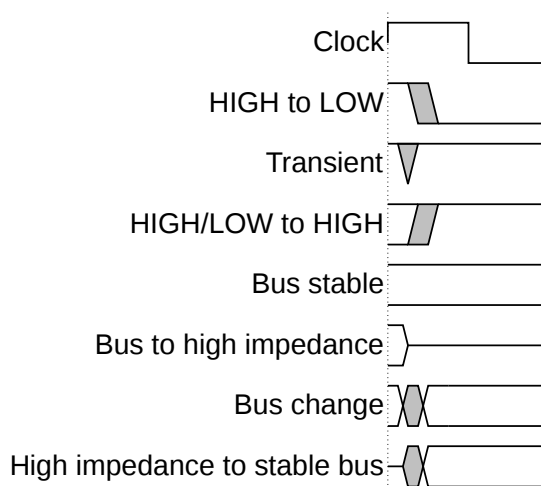


Figure 1: Key to timing diagram conventions

Timing diagrams sometimes show single-bit signals as HIGH and LOW at the same time and they look similar to the bus change shown in [Figure 1](#) diagram conventions. If a timing diagram shows a single-bit signal in this way, then its value does not affect the accompanying description.

Signals

The signal conventions are:

Signal level The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means:

- HIGH for active-HIGH signals.
- LOW for active-LOW signals.

Lowercase n At the start or end of a signal name denotes an active-LOW signal.

Lowercase x At the second letter of a signal name, denotes a collective term for both Read and Write. For example, **AxCACHE** refers to both the **ARCACHE** and **AWCACHE** signals.

Numbers

Numbers are normally written in decimal. Binary numbers are preceded by `0b`, and hexadecimal numbers by `0x`. Both are written in a `monospace` font.

Additional reading

This section lists publications by Arm and by third parties.

See Arm Developer, <http://developer.arm.com> for access to Arm documentation.

Arm publications

- *AMBA[®] AXI Protocol Specification* (ARM IHI 0022).
- *AMBA[®] DTI Protocol Specification* (ARM IHI 0088).
- *AMBA[®] CHI Architecture Specification* (ARM IHI 0050).
- *AMBA[®] CHI Chip-to-Chip (C2C) Architecture Specification* (ARM IHI 0098).
- *Arm[®] System Memory Management Unit Architecture Specification* (ARM IHI 0070).
- *Arm[®] Architecture Reference Manual for A-profile Architecture* (ARM DDI 0487).

Other publications

- *PCIe Express Base Specification, Revision 7, PCI-SIG*
- *Compute Express Link (CXL), CXL Consortium*

Feedback

Arm welcomes feedback on its documentation.

Feedback on this specification

If you have any comments or queries about our documentation, create a ticket at <https://support.developer.arm.com>.

As part of the ticket, please include:

- The title (AMBA® LTI Protocol Specification).
- The number (ARM IHI 0089 Issue D).
- The section name to which your comments refer.
- The page number(s) to which your comments refer.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.

Inclusive terminology commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive. Arm strives to lead the industry and create change.

Previous issues of this document included language that can be offensive. We have replaced this language. To report offensive language in this document, email terms@arm.com.

Part B

Specification

Chapter B1

Introduction

This chapter introduces the AMBA[®] LTI protocol specification. It contains the following sections:

- [*B1.1 About the LTI protocol*](#)
- [*B1.2 Use cases*](#)
- [*B1.3 Differences between DTI and LTI*](#)
- [*B1.4 Supported translation flows*](#)

B1.1 About the LTI protocol

The AMBA® LTI protocol specification aligns with the SMMU architecture and complements AMBA® *Distributed Translation Interface* (DTI) to provide higher performance and more efficient translation services.

LTI is a point-to-point protocol and defines the communication between an IO Manager and a *Translation Buffer Unit* (TBU). LTI enables devices to directly request a translation for each transaction while leaving the TBU to manage the *Translation Lookaside Buffer* (TLB). This enables translations to be requested before ordering requirements are met and avoiding the need to pass transactions through the TBU. This provides improved performance and reduced silicon area.

This specification describes the LTI protocol and the components of an LTI-compliant implementation. The LTI protocol is used by implementations of the *Arm® System Memory Management Unit Specification* (SMMU).

B1.2 Use cases

A system incorporating an LTI-based *System Memory Management Unit* (SMMU) might have the following components:

- A client device whose transactions require translation.
- A *Bus Interface Unit* (BIU), which fetches a translation for each transaction using LTI.
- A *TLB Unit* (TLBU) caches translations in a TLB. TLBU receives translation requests using LTI, and if the requested translation is not available in its TLB, it requests translations from a TCU using DTI.
- A *Translation Control Unit* (TCU), which calculates translations, reading translation tables when required.

Typically, the BIU and TLBU are packaged together as a TBU. Alternatively, the TBU might consist of just the TLBU.

A device can be integrated in one of the following ways:

- [B1.2.1 In-line integration](#)
- [B1.2.2 Lookaside integration](#)
- [B1.2.3 Cached integration](#)

B1.2.1 In-line integration

When in-line integration is implemented, every transaction passes through the SMMU, typically using AXI. The device is not aware of translation. [Figure B1.1](#) shows the in-line integration option.

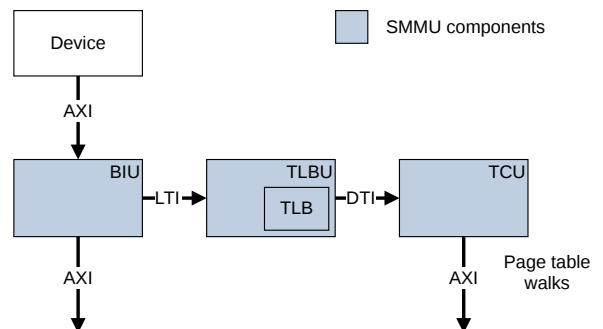


Figure B1.1: In-line SMMU Integration

B1.2.2 Lookaside integration

When lookaside integration is implemented, a translation request is made over LTI for each transaction, but the transaction data does not pass through the SMMU. Transaction completion is signaled to the TLBU using the LTI interface. This option is more complex than in-line integration, and enables the device to connect to the system using its native interface. [Figure B1.2](#) shows the lookaside integration option.

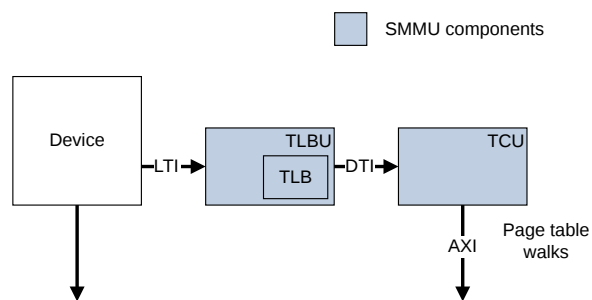


Figure B1.2: Lookaside SMMU integration

B1.2.3 Cached integration

When cached integration is implemented, the device requests translations over DTI when required and caches them locally in an integrated TLB. The device must support invalidation messages. This option is the most complex, but gives the device control over how translations are cached, and can be the most efficient option for devices with specific caching requirements. [Figure B1.3](#) shows the cached integration option.

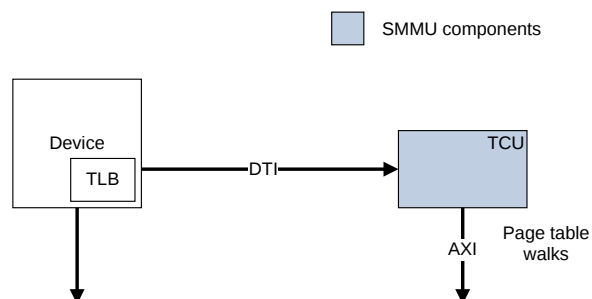


Figure B1.3: Cached SMMU integration

B1.3 Differences between DTI and LTI

Table B1.1 shows the differences between the DTI and LTI protocols.

Table B1.1: LTI and DTI comparison

Category	LTI	DTI
Topology	Designed for connecting to a single, local TLBU over a short distance.	Designed for connecting multiple TBUs to a TCU over a longer distance.
Caching and invalidation	Configurable as one of the following behaviors: – Translations can be cached. The LTI Manager must support LT messages to invalidate translations and return the corresponding LC messages. – Translations must be used immediately and must not be cached. No invalidation is required.	Translations can be cached. The DTI TBU or PCIe <i>Root Port RP</i> must support invalidation messages to invalidate translations that are previously returned.
Transport	AXI-like channel-based protocol gives dedicated signals for each data field to maximize throughput and avoid protocol conversion latency.	Message-based protocol can easily be passed over generic transports.
Mapping translations to transactions	Provides the translated transaction information directly, making it simple to use.	Requires the TBU or PCIe RP to follow rules on how to map cache entries on to each transaction.
Ordering	Supports ordering of translations to match transaction ordering, or out-of-order operation.	Supports fully out-of-order operation.

B1.4 Supported translation flows

Several translation flows are supported by LTI.

B1.4.1 Stall flow

When the Stall flow is used, software can configure the SMMU to take one of the following actions when a translation fault occurs:

- Immediately terminate the transaction and optionally record an error record that informs software that the transaction was terminated.
- Stall the translation and inform software that the translation is stalled. Software can then terminate the transaction, or update the translation tables and retry the translation. The LTI Manager is not aware of the stall.

Stall flow enables software to manage translation faults and demand paging without the client device being aware. However, it has some limitations:

- The LTI Manager can see long translation times, potentially triggering timeouts.
- Due to the dependence of software activity, the Stall flow can cause deadlocks in some systems.

For example, Stall flow is not recommended for use with PCIe, because of dependencies between outgoing transactions to PCIe from a CPU, and incoming transactions from PCIe through the SMMU.

Stall flow is the most common flow for LTI Managers.

Enabling the Stall flow does not necessarily cause a stall when a translation fault occurs. Stalls only occur when enabled by software. Software does not normally enable stalling for PCIe endpoints.

B1.4.2 ATST flow

The *Address Translation Service Translated* (ATST) flow is a flow that is used by PCIe Root Ports only. The flow is:

1. A PCIe endpoint requests a translation over ATS, which is passed to the SMMU by the PCIe RP using the DTI-ATS protocol.
2. The SMMU responds to the ATS request over DTI-ATS, which is passed back to the PCIe endpoint by the PCIe RP. If a translation fault occurs, then the response indicates the condition, and does not make any software-visible record.
3. The PCIe endpoint uses the ATS translation to translate the address of transaction, and then issues a transaction that is marked as ATS Translated.

An ATS Translated transaction uses the ATST flow. In all other respects, it is translated the same way as any other transaction. Normally this translation is fast because it is already translated by ATS, but some additional translation might still occur. For example, the SMMU can be configured to perform stage 1 translation when an ATS request is made, and perform stage 2 translation when the ATS Translated transaction is presented to the SMMU.

ATS translations are cached in the PCIe endpoint *Address Translation Cache* (ATC), for future use. These translations can be invalidated by ATS invalidation messages, which is conveyed over DTI-ATS.

If a translation fault occurs, the PCIe endpoint can issue a page request using the *Page Request Interface* (PRI), before retrying the ATS translation request if the PRI request is successful. PRI is an extension to ATS, and is also conveyed to the SMMU using DTI-ATS. The fault is not visible to LTI.

B1.4.3 NoStall flow

NoStall flow is used by a Manager that is not able to be stalled. If a translation fault occurs, the transaction is terminated, even if software has configured the device to be stalled when a translation fault occurs.

B1.4.4 PRI flow

The PRI flow is designed for use with a PCIe-enumerated endpoint. The device uses the LTI PRI flow to enable software to respond to translation faults without risking deadlock. From a software point of view:

- The device is a PCIe endpoint.
- The device uses the ATS protocol to fetch translations.
- When a translation fault occurs, the device makes a page request using PRI.

The behavior of the LTI PRI flow is as follows:

- The SMMU ATS features are not required and not used, even though ATS is enabled in software. Transactions are translated on-demand.
- If a translation fault occurs, no error is reported to software by the SMMU. Instead, a PRI fault response is returned to the LTI Manager.
- When the LTI Manager receives a PRI fault response, it uses a DTI-ATS connection to issue a page request. If the PRI response is successful, then the LTI Manager retries the transaction.

A device using this flow uses DTI-ATS for PRI requests only and does not make any ATS requests. In DTIv2, a device can implement a DTI-ATS connection that just performs PRI requests and does not receive ATS invalidation messages.

A device must be assigned page request credits by system software before it can issue PRI requests. These credits are not visible to the LTI or DTI protocols and are managed by PCIe software.

It is recommended that a device with an AXI interface with Ordered Write Observation does not use **AxMMUFLOW = PRI** as this could break the producer-consumer model.

B1.5 Translation cache

A translation cache allows translations to be reused within larger granules and allows the Subordinate to have a mechanism that explicitly revokes the translation on the LT Channel. This enables the Managers to build simple LTI caches that can be used to filter redundant translations and decrease LTI bandwidth requirements for certain workloads with spatial and temporal locality, for example streaming workloads, and large pages. The TLBU can indicate that the translation applies to a range of addresses greater than 4KB and whilst only requiring one cache entry.

If a Manager does not include a translation cache, translations are only permitted to be reused within a 4KB boundary for a short period of time by delaying the completion of the translation.

Chapter B2

Channels

This chapter describes LTI information flow. It contains the following sections:

- [*B2.1 Transaction flow*](#)
- [*B2.2 Virtual Channels*](#)
- [*B2.3 Flow control*](#)
- [*B2.4 Reserved encodings*](#)
- [*B2.5 Signal validity*](#)

B2.1 Transaction flow

Table B2.1 shows the three LTI channels. For each channel, the TX is the component that sends a message on the channel and the RX is the component that receives the message.

Table B2.1: LTI interface channels

Channel name	Channel prefix	Channel TX	Channel RX
LTI request	LA	Manager	Subordinate
LTI response	LR	Subordinate	Manager
LTI completion	LC	Manager	Subordinate
LTI tag	LT	Subordinate	Manager

Additionally, interface management signals are included with the prefix LM.

In this specification, the term, Lx, is used to collectively refer to LA, LR, LT, and LC.

A complete LTI transaction consists of a message on all three channels in sequence:

1. The Manager sends a request on the LA channel.
2. The Subordinate sends a response on the LR channel. Only after receiving the transaction can the translated transaction be issued downstream.
3. The Manager sends a completion on the LC channel, once the downstream transaction is complete.

There are no flows where any of these three stages can be skipped.

The relationship between LTI messages and the translated transaction are shown in Figure B2.1.

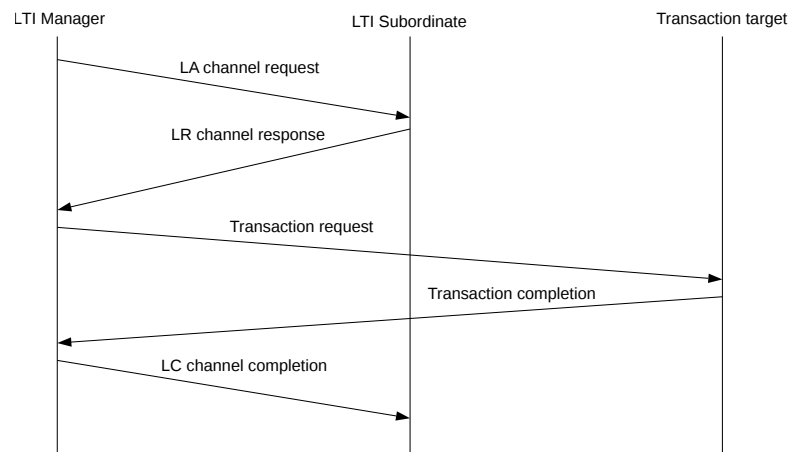


Figure B2.1: Relationship of messages

The LTI Subordinate does not correlate the completion messages to a specific LTI transaction, but instead counts the number of completions corresponding to each completion tag. The purpose of the Completion channel is to enable the LTI Subordinate to determine when all transactions with a completion tag have completed. This checking of the number of completions for a tag is part of the translation invalidation process. There are two completion tags to enable invalidation to take place without stopping the transaction flow.

An LTI response is permitted in the same cycle that the corresponding request is made. This response timing enables tightly coupled TLBs with low latency.

An LTI completion is not permitted in the same cycle that the corresponding response is returned. An LTI response can be followed by its completion in the following cycle or later.

All signals are driven by the channel TX, except **LxCREDIT** signals, which are driven by the channel RX.

An LTI transaction is considered to be outstanding from the cycle in which the LA request is used until the cycle in which the LC completion is issued.

B2.2 Virtual Channels

The LA and LR channels can be composed of multiple Virtual Channels. The LA and LR channels must support the same number of Virtual Channels.

The LC channel does not support multiple Virtual Channels. The LTI Subordinate must always be able to provide a credit on the LC channel without dependence on progress of other channels.

In an LTI transaction, messages on the LA and LR channels must use the same *Virtual Channel* (VC).

Because LC messages are not associated with a particular VC, it is necessary that any counter in the LTI Subordinate that is counting outstanding completions does not overflow. The LTI Subordinate must be able to track at least 65535 translation responses awaiting a completion. When this number is exceeded, the LTI Subordinate is permitted to introduce dependencies between Virtual Channels, by not returning a translation response until translation completions from previous translations are returned.

The intent of Virtual Channels is to enable one VC to progress when another is blocked, to avoid deadlock scenarios.

Components implementing LTI must ensure that if progress is blocked on one VC it does not result in progress being blocked on a different VC.

In some cases, the forward progress properties of a single Virtual Channel is sufficient for an implementation.

Within a VC, LTI must make independent forward progress on all LTI transactions when:

1. The LA STALL flow is not used on any request.
2. The LTI Manager is able to provide a credit to the LR channel without dependence on the progress of any other transaction.
3. The total number of translations that have sent an LTI request but have not sent the LTI completion must not exceed the completion tracking limit of the LTI Subordinate.

B2.3 Flow control

Flow control in the LTI protocol has the following rules:

- Each channel includes **LxVALID** and **LxCREDIT** signals. If more than one VC is supported, the LA and LR channels include **LxVC** signals.
- Each cycle that **LxCREDIT**[*n*] is asserted grants one credit on VC *n* to the channel TX.
- The channel TX consumes a credit on VC *n* each cycle that **LxVALID** is asserted with **LxVC** = *n*.
- **LxVALID** cannot be asserted with **LxVC** = *n* when the channel TX has zero credits for VC *n*.
- The maximum number of credits that the channel RX can grant the channel TX for each VC is 15.

In addition, there must not be combinatorial paths between **LxCREDIT** and other signals on a channel in either direction, such as **LxVALID**. This restriction has the following consequences:

- A credit cannot be used by **LxVALID** in the same cycle that it is granted by **LxCREDIT** when there are no other credits granted.
- A credit cannot be returned on **LxCREDIT** in the same cycle that it is used by **LxVALID** when the maximum number of credits that are permitted to be granted is reached.

B2.4 Reserved encodings

Where encodings are given, unused encodings are Reserved.

The use of a Reserved encoding in a field that is not ignored is a protocol error and can lead to UNPREDICTABLE behavior.

B2.5 Signal validity

A signal can be defined as not valid within this specification. When a signal is not valid:

- The channel TX can drive it to any value.
- The channel RX must ignore its value.

Chapter B3

Properties

This chapter describes the set of LTI properties that specifies the supported behavior and interface signaling requirements. It contains the following section:

- [B3.1 LTI properties](#)

B3.1 LTI properties

Table B3.1 describes the LTI properties.

If a property is not defined on the interface, the default value is taken.

Table B3.1: LTI properties

Name	Data type	Description
LTI_VC_COUNT	Integer	Number of Virtual Channels. The legal values are 1 to 8.
LTI_ID_WIDTH	Integer	Width of translation request ID. The legal values are 0 to 32.
LTI_SID_WIDTH	Integer	Width of StreamID. Higher-order bits of StreamID might be statically defined by the LTI Subordinate, for example using a tie-off. The legal values are 0 to 32.
LTI_SSID_WIDTH	Integer	Width of SubstreamID. The legal values are 0 to 20.
LTI_OG_WIDTH	Integer	Width of order group. The legal values are 0 to 32.
LTI_TLBLOC_WIDTH	Integer	Width of TLB location. The legal values are 0 to 128.
LTI_LOOP_WIDTH	Integer	Width of loopback signals. The legal values are 0 to 256.
LTI_LRADDR_WIDTH	Integer	Width of translated address. The legal values must be one of the following: – 32 – 36 – 40 – 42 – 44 – 48 – 52
LTI_LAUSER_WIDTH	Integer	Width of address channel user signals. The legal values are 0 to 128.
LTI_LRUSER_WIDTH	Integer	Width of response channel user signals. The legal values are 0 to 128.
LTI_LCUSER_WIDTH	Integer	Width of completion channel user signals. The legal values are 0 to 128.
LTI_LTUSER_WIDTH	Integer	Width of tag channel user signal. The legal values are 0 to 128. The default value is 0. The value must be 0 if the LTI_CACHE_SUPPORT property is False.
LTI_MECID_WIDTH	Integer	Width of <i>Memory Encryption Context Identifier</i> (MECID). The legal values must be one of the following: – 0 – 16 When LTI_GPC is False, LTI_MECID_WIDTH must be 0.

Continued on next page

Table B3.1 – Continued from previous page

Name	Data type	Description
LTI_MPAM_WIDTH	Integer	Width of MPAM signals. The legal values must be one of the following: MPAM_9_1 This is the default value. The interface is enabled for MPAM and includes LRMPAM. The width of PARTID is 9 and PMG is 1. MPAM_12_1 The interface is enabled for MPAM and includes LRMPAM. The width of PARTID is 12 and PMG is 1. False MPAM is not supported and LRMPAM is not present. When LTI_MPAM_SUPPORT == False, the value is 0. When LTI_MPAM_SUPPORT == MPAM_9_1 and LTI_GPC == False, the value is 11. When LTI_MPAM_SUPPORT == MPAM_9_1 and LTI_GPC == True, the value is 12. When LTI_MPAM_SUPPORT == MPAM_12_1 and LTI_GPC == False, the value is 14. When LTI_MPAM_SUPPORT == MPAM_12_1 and LTI_GPC == True, the value is 15. The default value is 0.
LTI_PAS_WIDTH	Integer	Width of the physical address space signals. The legal values are 0 to 3. The default value is 0. LTI_PAS_WIDTH must be non-zero for an LTI-D interface. When LTI_GPC is True, LTI_PAS_WIDTH must be greater than 1. When LTI_GDI is True, LTI_PAS_WIDTH must be greater than 2.
LTI_CTAG_WIDTH	Integer	Width of the LTI completion tag. The legal values are 1 to 16. The default value is 1. Wider CTAG provides a larger pool of completion tags to be used for LTI transactions. This allows more precise invalidation mechanisms to be implemented in the TLBU and helps to reduce the synchronization latency.
LTI_GPC	Boolean	Indicates whether the SMMU is configured to perform <i>Granule Protection Checks</i> (GPC), as part of Realm Management Extension (RME). The legal values must be one of the following: True The interface supports GPC. False The interface does not support GPC. The default value is False.
LTI_GDI	Boolean	Indicates whether Granular Data Isolation (GDI) is supported. The legal values must be one of the following: True The interface supports GDI. NSP and SA address spaces can be used. False The interface does not support GDI. The default value is False. When LTI_GPC is False, LTI_GDI must be False.
LTI_MMU	Boolean	Indicates whether SMMU is configured to perform stage 1 and stage 2 translations. The legal values must be one of the following: True The interface supports stage 1 and stage 2 translations. False The interface does not support stage 1 and stage 2 translations. The default value is True. LTI_GPC and LTI_MMU must not both be False.
LTI_MPAM_SUPPORT	Enumeration	Indicate whether the interface supports MPAM. The legal values must be one of the following: MPAM_9_1 The interface is enabled for MPAM and includes LRMPAM. The width of PARTID is 9 and PMG is 1. MPAM_12_1 The interface is enabled for MPAM and includes LRMPAM. The width of PARTID is 12 and PMG is 1. False MPAM is not supported and LRMPAM is not present. The default value is MPAM_9_1. See Table B12.3 for more information on interoperability between an LTI Manager and Subordinate.

Continued on next page

Table B3.1 – Continued from previous page

Name	Data type	Description
LTI_CACHE_SUPPORT	Boolean	Indicates whether the translation cache is supported. The legal values must be one of the following: True The interface supports translation cache. False The interface does not support translation cache. The default value is False.
LTI_PASUNKNOWN_PRESENT	Boolean	Indicates whether the LAPASUNKNOWN signal is present on the interface. The legal values must be one of the following: True The LAPASUNKNOWN signal is present on the interface. The Realm context request can use the LAPASUNKNOWN to indicate whether it includes a meaningful PAS. False The LAPASUNKNOWN signal is not present on the interface. The default value is False. When LTI_GPC is False or LTI_MMU is False, LTI_PASUNKNOWN_PRESENT must be False.
LTI_OUTER_CACHEABLE_MODE	Boolean	Affects the computation of the transaction attributes. The legal values must be one of the following: True Transaction attributes are in outer cacheable mode. False Transaction attributes are not in outer cacheable mode. The default value is False.
LTI_CHECK_TYPE	Enumeration	Defines the protection scheme employed on the interface. The legal values must be one of the following: Odd_Parity_Byte_All Odd parity checking included for all signals. Each bit of the parity signal generally covers up to 8 bits. However, a parity bit can cover more than 8 bits if required by the configuration. False No checking signals on the interface. The default value is False.

When the value of a property results in a signal being 0 bits in width, that signal is omitted from the interface.

Table B3.2 shows how translations are affected by the combinations of properties LTI_GPC and LTI_MMU, and the signal **LAMMUV**.

Table B3.2: Legal combinations of properties LTI_GPC and LTI_MMU

LTI_GPC	LTI_MMU	Interface usage
True	True	RME is supported. When LAMMUV is HIGH, the SMMU performs stage 1 and stage 2 translations, as well as GPC. When LAMMUV is LOW, SMMU only performs a GPC.
True	False	RME is supported. The SMMU only performs GPC. LAMMUV must be LOW.
False	True	RME is not supported. When LAMMUV is HIGH, the SMMU only performs stage 1 and stage 2 translations. When LAMMUV is LOW, the SMMU does not perform stage 1 and stage 2 translations, nor a GPC.

Chapter B4

Request channel

This chapter defines the LTI request (LA) channel. It contains the following sections:

- [B4.1 Signals](#)
- [B4.2 Transaction types](#)
- [B4.3 Transaction attributes](#)
- [B4.4 Transaction scope](#)

B4.1 Signals

Table B4.1 describes the signals in the LA channel.

Table B4.1: Request channel signals

Signal	Category	Width	Description
LAVVALID	Transport	1	Channel valid. When this signal is LOW, other TX signals on the LA channel are not valid.
LAVC	Transport	$\text{ceil}(\log_2(\text{LTI_VC_COUNT}))$	Virtual Channel number.
LACREDIT	Transport	LTI_VC_COUNT	Channel credit grant. This is an RX signal that flows in the other direction from the other LA channel signals. It is not affected by LAVVALID .
LAIID	Flow	LTI_ID_WIDTH	Translation request ID. The ID must not match the ID of a previous request on the same VC that has not yet returned its LR channel response, unless LAOGV = 1 in both requests and the value of LAOG is the same for both requests.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LAOGV	Flow	1	Order group valid. LAOGV has the following encodings: 0 No ordering required. 1 The responses of all requests with this LAOG value must be returned in order. There is no order requirement between requests on different Virtual Channels. When LATRANS = UNSPEC, LAOGV must be LOW. This signal is present even if LTI_OG_WIDTH = 0. In this case, there is a single order group, and LAOGV indicates whether each LTI request is ordered or unordered.
Note If an LTI Manager uses LAFLOW = Stall for transactions issued downstream with order requirements, it is recommended for the LTI Manager to set LAOGV = 1 and use LTI order group instead of using unordered LTI requests. This prevents reordering the LTI responses and avoids deadlock. It is permitted to use unordered LTI requests for transactions issued downstream without order requirements.			
LAOG	Flow	LTI_OG_WIDTH	Order group. When LAOGV is LOW, this signal is not valid.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LAFLOW	Context	2	Indicates the translation flow required. LAFLOW has the following values: 0: Stall The SMMU stall fault flow can be used for this request, when it is enabled. 1: ATST The transaction has already been translated by ATS. 2: NoStall If a translation fault occurs, then even if the SMMU has enabled stall faulting for this translation context, a fault response is returned without dependence on software activity. 3: PRI If a translation fault occurs, a fault response is returned indicating that a PRI request might resolve the fault. Architecturally, the request is treated as an ATS request, and translation faults do not result in an event record. This option is for use by PCIe enumerated endpoints. For more information, see B1.4.4 PRI flow. If LAFLOW is not Stall, then the Subordinate must return the response in reasonable time, without dependence on software activity. It must not be Stall for PCIe Managers. When LAMMUV is 1 and LAPM is 1, LAFLOW must not be Stall. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LAMMUV	Context	1	Indicates whether SMMU translation is required. When LAMMUV is LOW, SMMU stage 1 and stage 2 translation is not performed. The GPC is always performed, if implemented, regardless of the LAMMUV signal. When LAMMUV is HIGH, transactions that support Memory Tagging are not supported. If LTI_MMU is False, LAMMUV must be LOW.
LASECSID	Context	$LTI_GPC == \text{True} ? 2:1$	StreamID security level. When LTI_GPC is False, LASECSID has the following encodings: 0b0 Non-secure StreamID 0b1 Secure StreamID When LTI_GPC is True, LASECSID has the following encodings: 0b00 Non-secure StreamID 0b01 Secure StreamID 0b10 Realm StreamID 0b11 Reserved When LAFLOW is ATST, this signal must be Non-secure or Realm. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.
LASID	Context	LTI_SID_WIDTH	StreamID. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.
LASSIDV	Context	$LTI_SSID_WIDTH > 0 ? 1:0$	SubstreamID valid. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True. When LAFLOW = ATST, the TBU implementation may ignore PASID. In such an implementation, the TBU must treat LASSIDV, LASSID, LAPRIV, and LAINST as 0 when LAFLOW = ATST.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LASSID	Context	LTI_SSID_WIDTH	SubstreamID. When LASSIDV is LOW, this signal must be 0. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.
LAPM	Context	1	Protected Mode attribute for the Non-secure transaction. LAPM has the following encodings: 0b0 The PM attribute is Clear on the transaction. 0b1 The PM attribute is Set on the transaction. When LAMMUV is 0, this signal is not valid. When LASECSID is Secure or Realm, LAPM must be 0. LAPM is only present when LTI_GDI is True and LTI_MMU is True.
LAPASUNKNOWN	Transaction	1	Indicates whether LAPAS carries a meaningful PAS. LAPASUNKNOWN has the following encodings: 0b0 LAPAS indicates a meaningful PAS. 0b1 LAPAS does not indicate a meaningful PAS. When LAMMUV is 0, this signal is not valid. When LAMMUV is 1 and LASECSID is Secure or Non-secure, this signal must be 0. LAPASUNKNOWN is only present when LTI_PASUNKNOWN_PRESENT is True.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LAPAS	Transaction	LTI_PAS_WIDTH	Indicates the PAS in the translation request. LAPAS has the following encodings: Secure 0b000 Non-secure 0b001 Root 0b010 Realm 0b011 System Agent (SA) 0b100 Non-secure Protected (NSP) 0b101 Others Reserved When LTI_PAS_WIDTH is 1, this signal must be Non-secure or Secure. When LTI_PAS_WIDTH is 2, this signal must be Non-secure, Secure, Root, or Realm. When LAMMUV is 0: – If LTI_GPC is False, LAPAS must be Non-secure or Secure. – If LTI_GPC is True and LTI_GDI is False, LAPAS must be Non-secure, Secure, Root, or Realm. When LAMMUV is 1 and LASECSID is Non-secure, this signal must be Non-secure. When LAMMUV is 1 and LASECSID is Secure, this signal must be Non-secure or Secure. When LAMMUV is 1 and LASECSID is Realm, this signal must be Non-secure or Realm. When LAMMUV is 1 and LAPASUNKNOWN is 1, this signal must be Realm.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LAPRIV	Transaction	1	Identifies an access as Privileged or Unprivileged. LAPRIV has the following encodings: 0b0 Unprivileged access. 0b1 Privileged access. LAPRIV must be 0 if any of the following combinations are True: – LATRANS is SPEC or UNSPEC. – LAFLOW is ATST and LASSIDV is 0. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.
LAINST	Transaction	1	Indicates whether the access is data or instruction. LAINST has the following encodings: 0 Data access. 1 Instruction access. LAINST must be 0 if any of the following combinations are True: – LATRANS is W, RW, SPEC, UNSPEC, W-CMO, DHCMO, DCP, or W-DCP. – LAFLOW is ATST and LASSIDV is 0. This signal is not valid when LAMMUV is LOW. This signal is present only when LTI_MMU is True.
LAADDR	Transaction	LTI_MMU == True: 64 LTI_MMU == False: LTI_LRADDR_WIDTH	Address. When LTI_MMU is True, this signal is always 64 bits because virtual addresses are always 64 bits, even if the system address bus is narrower. LAADDR[11:0] does not affect the translation result, but is used to provide information to software when a translation fault occurs.
LATRANS	Transaction	4	Indicates the type of the transaction that the LTI request is translating. See B4.2 Transaction types.
LAATTR	Transaction	4	Attributes for the untranslated transaction. See B4.3 Transaction attributes.

Continued on next page

Table B4.1 – Continued from previous page

Signal	Category	Width	Description
LALOOP	Impdef	LTI_LOOP_WIDTH	IMPLEMENTATION DEFINED loopback signaling.
LATLBLOC	Impdef	LTI_TLBLOC_WIDTH	Indicates the location to access in the TLB. This signal is IMPLEMENTATION DEFINED. The intended use of this signal is to control allocation and lookup in a TLB. For example, an implementation might guarantee that if a request is made with a particular value of LATLBLOC , and this request is followed by another request with the same value of LATLBLOC , any translation that is cached from the first request is available to be used by the second request. Alternatively, LATLBLOC might be used to indicate a portion of a TLB to use. This specification does not provide any guarantees about any such functionality.
LAUSER	Impdef	LTI_LAUSER_WIDTH	IMPLEMENTATION DEFINED additional signaling.
LAIDENT	Context	1	Indicates an identity translation is required. This signal can only be HIGH if LAFLOW = ATST. When LAMMUV is LOW, this signal is not valid. This signal is only present if LTI_MMU is True.

B4.2 Transaction types

Table B4.2 describes the valid values of **LATTRANS** and how those values correspond to types of translated transaction.

Table B4.2: Request channel transactions

Encoding	Name	Definition
0	SPEC	Speculative request. Intended to prefetch a translation for later use and is not used by a transaction.
Note The <i>AMBA® DTI Protocol Specification</i> permits speculative translation requests being used for speculative read transactions in some circumstances. The LTI SPEC transaction type cannot be used for speculative read transactions. The LTI SPEC transaction type prefetches a translation into translation caches and the translation that is returned cannot be used by transactions. If an LTI Manager requires a speculative read transaction that is guaranteed not to fault, then it should issue an LTI request with LATTRANS = R, LAFLOW = PRI.		
1	R	Read.
2	W	A write, or a write with Directed Cache Prefetch without specifying a target cache to stash the written data.
3	RW	Read and Write. Intended for atomic operations. Operations that perform any form of read-modify-write sequence on data in memory are defined as atomic operations, even if no read data is returned to the upstream Manager. For example, the AXI AtomicStore transaction type is atomic and must use LATTRANS = RW, even though it does not return read data.
4	CMO	Cache Maintenance Operation.
5	R-CMO	Read with Cache Maintenance Operation. R-CMO is a read that also performs a Cache Maintenance Operation.
6	W-CMO	Write with Cache Maintenance Operation. W-CMO is a write that also performs a Cache Maintenance Operation.
7	UNSPEC	Hint that the translation can be deallocated and is no longer required.
8	DCMO	Destructive Cache Maintenance Operation. Invalidated cache lines that are dirty are not required to be cleaned before being invalidated. DCMO operations require extra permission because they can enable old versions of a memory location to be recovered after they have been overwritten.
9	R-DCMO	Read with DCMO.
11	DHCMO	Hint that the cache line is no longer required. Downstream caches are permitted but not required to invalidate the line. If the line is invalidated and dirty, it is not required to be cleaned before being invalidated. Unlike a DCMO, the cache maintenance operation can be ignored with a DHCMO.
12	DCP	Directed Cache Prefetch. DCP is a request to prefetch data into a particular cache.
14	W-DCP	A write with Directed Cache Prefetch which specifies a target cache to stash the written data.

B4.3 Transaction attributes

Table B4.3 shows the valid encodings of **LAATTR** and how those values represent the untranslated transaction attributes.

Table B4.3: Request channel transaction attributes

Encoding	Type when LTI_OUTER_CACHEABLE_MODE is	
	True	False
0	Device-nGnRnE	Device-nGnRnE
1	Device-nGnRE	Device-nGnRE
2	Device-nGRE	Device-nGRE
3	Device-GRE	Device-GRE
4	Normal Non-cacheable	Normal Non-cacheable
5	Reserved ^a	Normal Inner Non-cacheable Outer Cacheable
6	Snoopable Write-Back No-allocate	Snoopable Write-Back No-allocate
7	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
14	Non-snoopable Write-Back No-allocate	Non-snoopable Write-Back No-allocate
15	Non-snoopable Write-Back Allocate	Non-snoopable Write-Back Allocate

^a When LTI_OUTER_CACHEABLE_MODE is True, the encoding 5 is Reserved for **LAATTR** and **LRATTR**.

All other encodings are Reserved.

Most transactions should use the value 7 for **LAATTR**, Snoopable Write-Back Allocate. This value is the common type for accessing normal memory locations when the translation tables have not selected a different set of transaction attributes.

The term ‘Normal Write-Back’ collectively represents Snoopable Write-Back and Non-snoopable Write-Back.

Table B4.4 shows the attribute restrictions for specific transaction types.

Table B4.4: Request attribute restrictions for specific transaction types

LATRANS	Legal values of LAATTR when LTI_OUTER_CACHEABLE_MODE is	
	True	False
CMO, DCMO	Snoopable Write-Back No-allocate	Snoopable Write-Back No-allocate
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
	-	Non-snoopable Write-Back No-allocate
	-	Non-snoopable Write-Back Allocate
DHCMO, DCP	Snoopable Write-Back No-allocate	Snoopable Write-Back No-allocate
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate

Continued on next page

Table B4.4 – Continued from previous page

LATRANS	Legal values of LAATTR when LTI_OUTER_CACHEABLE_MODE is	
	True	False
W-DCP, R-CMO, or R-DCMO	Non-snoopable Write-Back No-allocate	Non-snoopable Write-Back No-allocate
	Non-snoopable Write-Back Allocate	Non-snoopable Write-Back Allocate
	Snoopable Write-Back No-allocate	Snoopable Write-Back No-allocate
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate

Note

No-allocate is treated as Allocate for CMO, DCMO, and DHCMO.

There is no attribute restriction for **LAATTR** when **LATRANS** is R, W, RW, SPEC, W-CMO, or UNSPEC.

Note

In Issue A and Issue B, W-CMO had attribute restrictions on **LAATTR**. This is no longer required for Issue C and onwards.

B4.4 Transaction scope

An LTI translation can only be used for accesses to data within a single page with the page size determined as follows:

- When LTI_CACHE_SUPPORT is False, the page size is 4KB.
- When LTI_CACHE_SUPPORT is True, the page size is the size indicated by **LRSIZE**.

If a memory transaction accesses data in more than one page, a separate LTI request must be made for the part of the transaction in each page.

B4.5 Context and PAS combinations

Table B4.5 shows the possible combinations of LAMMUV, LASECSID, LAFLOW, LAPM, LAPASUNKNOWN and LAPAS signals. All other combinations are not legal.

Table B4.5: Meaning of the combinations of LAMMUV, LASECSID, LAFLOW, LAPM, LAPASUNKNOWN and LAPAS

LAMMUV	LASECSID	LAFLOW	LAPM	LAPASUNKNOWN	LAPAS	Meaning
0	-	-	-	-	Secure	NoStreamID, Secure PAS
0	-	-	-	-	Non-secure	NoStreamID, Non-secure PAS
0	-	-	-	-	Root	NoStreamID, Root PAS
0	-	-	-	-	Realm	NoStreamID, Realm PAS
0	-	-	-	-	SA	NoStreamID, System Agent PAS
0	-	-	-	-	NSP	NoStreamID, Non-secure Protected PAS
1	Non-secure	PRI NoStall Stall	0	0	Non-secure	Untranslated Non-secure context
1	Non-secure	PRI NoStall	1	0	Non-secure	Untranslated Non-secure context, Protected Mode
1	Non-secure	ATST	0	0	Non-secure	Translated Non-secure context
1	Non-secure	ATST	1	0	Non-secure	Translated Non-secure context, Protected Mode
1	Secure	PRI NoStall Stall	0	0	Secure	Untranslated Secure context, Secure PAS
1	Secure	PRI NoStall Stall	0	0	Non-secure	Untranslated Secure context, Non-secure PAS
1	Realm	PRI NoStall Stall	0	0	Non-secure	Untranslated Realm context, Non-secure PAS
1	Realm	PRI NoStall Stall	0	0	Realm	Untranslated Realm context, Realm PAS
1	Realm	PRI NoStall Stall	0	1	Realm	Untranslated Realm context, no PAS expectation
1	Realm	ATST	0	0	Non-secure	Translated Realm context, Non-secure PAS
1	Realm	ATST	0	0	Realm	Translated Realm context, Realm PAS
1	Realm	ATST	0	1	Realm	Translated Realm context, no PAS expectation

Chapter B5

Response channel

This chapter defines the LTI response (LR) channel. It contains the following sections:

- [B5.1 Signals](#)
- [B5.2 LRRESP details](#)
- [B5.3 Attribute restrictions for specific transaction types](#)

B5.1 Signals

Table B5.1 describes the signals in the LR channel.

Table B5.1: Response channel signals

Signal	Category	Width	Description
LRVALID	Transport	1	Channel valid. When this signal is LOW, other TX signals on the LR channel are not valid.
LRVC	Transport	$\text{ceil}(\log_2(\text{LTI_VC_COUNT}))$	VC number.
LRCREDIT	Transport	LTI_VC_COUNT	Channel credit grant. LRCREDIT is an RX signal that flows in the other direction from other LR channel signals. It is not affected by LRVALID .
LRID	Flow	LTI_ID_WIDTH	Translation request ID. The value of LRID must match a translation request that has not yet had a response.
LRCTAG	Flow	LTI_CTAG_WIDTH	Translation completion tag. LRCTAG can be any value and must be returned by the LTI Manager with the completion.

Continued on next page

Table B5.1 – Continued from previous page

Signal	Category	Width	Description
LRRESP	Translation	3	Translation response.
			0: Success The translation was successful.
			1: Downgrade1 The translation was successful but the transaction type must be downgraded. The meaning of this for each transaction type is described in B5.2.2 Downgrade types .
			2: Downgrade2 The translation was successful but the transaction type must be downgraded. The meaning of this for each transaction type is described in B5.2.2 Downgrade types .
			4: FaultAbort The translation was not successful and the transaction must be terminated. The Manager should indicate to the upstream device that the transaction was not successful.
			5: TerminateRAZWI The translation was not successful and the transaction must be terminated. If possible, the LTI Manager should indicate to the Requester that the transaction was successful, by returning 0 if the data was a read, and ignoring the transaction if it was a write. Cache maintenance and prefetch effects of the transaction are ignored.
			6: FaultPRI The translation was not successful but it might be resolved by issuing a PRI request. The Manager should issue a PRI request, and if the response from that indicates success, retry the LTI request. For more information, see B1.4.4 PRI flow .

Continued on next page

Table B5.1 – Continued from previous page

Signal	Category	Width	Description														
			Restrictions that are associated with this field are described in more detail in B5.3 Attribute restrictions for specific transaction types .														
LRPAS	Translation	LTI_PAS_WIDTH	Indicates the PAS in the translated transaction. LRPAS encodings: <table><tr><td>0b000</td><td>Secure</td></tr><tr><td>0b001</td><td>Non-secure</td></tr><tr><td>0b010</td><td>Root</td></tr><tr><td>0b011</td><td>Realm</td></tr><tr><td>0b100</td><td>System Agent (SA)</td></tr><tr><td>0b101</td><td>Non-secure Protected (NSP)</td></tr><tr><td>Others</td><td>Reserved</td></tr></table> When LTI_PAS_WIDTH is 1, LRPAS must be Non-secure or Secure. When LTI_PAS_WIDTH is 2, LRPAS must be Non-secure, Secure, Root, or Realm. When LAMMUV is 0, LRPAS must equal LAPAS. When LAMMUV is 1 and LASECSID is Non-secure: – If LTI_GDI is True and LAPM is 1, LRPAS must be Non-secure or NSP. – If LTI_GDI is False or LAPM is 0, LRPAS must be Non-secure. When LAMMUV is 1 and LASECSID is Secure, LRPAS must be Non-secure or Secure. When LAMMUV is 1 and LASECSID is Realm, LRPAS must be Non-secure or Realm. When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, LRPAS is not valid.	0b000	Secure	0b001	Non-secure	0b010	Root	0b011	Realm	0b100	System Agent (SA)	0b101	Non-secure Protected (NSP)	Others	Reserved
0b000	Secure																
0b001	Non-secure																
0b010	Root																
0b011	Realm																
0b100	System Agent (SA)																
0b101	Non-secure Protected (NSP)																
Others	Reserved																
LRADDR	Translation	LTI_LRADDR_WIDTH	Translated address. The least significant 12 bits must equal the least significant 12 bits of LAADDR. These bits are included in the response to avoid them needing to be included in loopback and provided in the request. When LAMMUV is LOW, LRADDR must match LAADDR. When LAMMUV is HIGH and LAIDENT is HIGH, LRADDR must match LAADDR. When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, this signal is not valid.														

Continued on next page

Table B5.1 – Continued from previous page

Signal	Category	Width	Description
LRATTR	Translation	4	Translated transaction attributes. LRATTR uses the same encoding as LAATTR. The permitted values of LRATTR depend on LATRANS after any downgrades have been applied, using the same rules as stated in B5.3 Attribute restrictions for specific transaction types. For more information, see B5.2 LRRESP details and B5.2.2 Downgrade types. When LAMMUV is LOW: – When LATRANS is CMO, DCMO, DHCMO or SPEC, LRATTR must match LAATTR, with the exception of the allocation hint which must be Allocate for cases where it is meaningful. – When LATRANS is any other transaction, LRATTR must match LAATTR. When LAFLOW = ATST, LAIDENT = 1, and LAATTR = Snoopable Write-Back, LRATTR must be Snoopable Write-Back. When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, this signal is not valid.
LRHWATTR	Translation	4	IMPLEMENTATION DEFINED hardware attributes. When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, this signal is not valid. When LAMMUV is LOW, LRHWATTR must be 0^a.

Continued on next page

Table B5.1 – Continued from previous page

Signal	Category	Width	Description
LRMPAM	Translation	LTI_LRMPAM_WIDTH	MPAM information. The mapping of MPAM fields to the LRMPAM signal depends on the LTI_GPC property. When LTI_MPAM_SUPPORT is MPAM_9_1: – When LTI_GPC is False: – LRMPAM[0] MPAM_NS – LRMPAM[9:1] PARTID – LRMPAM[10] PMG – When LTI_GPC is True: – LRMPAM[1:0] MPAM_SP – LRMPAM[10:2] PARTID – LRMPAM[11] PMG When LTI_MPAM_SUPPORT is MPAM_12_1: – When LTI_GPC is False: – LRMPAM[0] MPAM_NS – LRMPAM[12:1] PARTID – LRMPAM[13] PMG – When LTI_GPC is True: – LRMPAM[1:0] MPAM_SP – LRMPAM[13:2] PARTID – LRMPAM[14] PMG When LAMMUV is HIGH and LTI_GPC is False: – When LASECSID is Non-secure, MPAM_NS must be Non-secure. – When LASECSID is Secure, MPAM_NS must be Non-secure or Secure. When LAMMUV is HIGH and LTI_GPC is True: – When LASECSID is Non-secure, MPAM_SP must be Non-secure. – When LASECSID is Secure, MPAM_SP must be Non-secure or Secure. – When LASECSID is Realm, MPAM_SP must be Non-secure or Realm. When LAMMUV is LOW: – When LTI_GPC is False, MPAM_NS is the PAS in LRPAS. – When LTI_GPC is True: – If LRPAS is Secure, Non-secure, Realm, or Root, MPAM_SP is the PAS in LRPAS. – If LRPAS is NSP, MPAM_SP is Non-secure. – If LRPAS is SA, MPAM_SP is Root. – PARTID is 0. – PMG is 0.

Continued on next page

Table B5.1 – Continued from previous page

Signal	Category	Width	Description
			When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, this signal is not valid. This signal is only present when LTI_MPAM_SUPPORT != False.
LRMECID	Translation	LTI_MECID_WIDTH	Memory Encryption Context Identifier (MECID). When LRPAS != Realm, this signal must be 0. When LAMMUV is LOW, LRMECID must be 0 ^b . When LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI, this signal is not valid.
LRSIZE	Translation	6	Indicates the aligned range of addresses that this translation is valid for. The range of address indicated by LRSIZE is given by $2^{LRSIZE+12}$ bytes. The value of LRSIZE must not be greater than (LTI_LRADDR_WIDTH-12). LRADDR bits within the range indicated by LRSIZE must match LAADDR . For example, if the value of LRSIZE is 4, then LRADDR [15:0] must match LAADDR [15:0]. This signal is not valid when LRRESP is FaultAbort, FaultPRI, or TerminateRAZWI. This signal is present only when LTI_CACHE_SUPPORT is True.
LRLOOP	Translation	LTI_LOOP_WIDTH	Loopback signaling. Must match the value of LALoop in the request.
LRUSER	Impdef	LTI_LRUSER_WIDTH	IMPLEMENTATION DEFINED additional signaling.

^a This rule does not apply to an LTI-B interface which has a different rule for **LRHWATTR** value when **LAMMUV** is LOW. For more details, see Issue B.

^b This rule does not apply to an LTI-B interface which has a different rule for **LRMECID** value when **LAMMUV** is LOW. For more details, see Issue B.

B5.2 LRRESP details

This section gives further information about **LRRESP**.

B5.2.1 Restrictions based on LATRANS

[Table B5.2](#) shows the permitted encodings of **LRRESP**, depending on the value of **LATRANS** in the request.

Table B5.2: Response channel LATRANS restrictions

LATRANS	LRRESP encodings permitted when	
	LTI_MMU is True and LAMMUV is HIGH	LTI_MMU is False or LAMMUV is LOW
SPEC	Success, TerminateRAZWI	Success, TerminateRAZWI
R	Success, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
W	Success, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
RW	Success, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
CMO	Success, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
R-CMO	Success, Downgrade1, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
W-CMO	Success, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
UNSPEC	TerminateRAZWI	TerminateRAZWI ^a
DCMO	Success, Downgrade2, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
R-DCMO	Success, Downgrade1, Downgrade2, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort
DHCMO	Success, TerminateRAZWI	Success, TerminateRAZWI
DCP	Success, TerminateRAZWI	Success, TerminateRAZWI
W-DCP	Success, Downgrade1, FaultAbort, TerminateRAZWI, FaultPRI	Success, FaultAbort

^a This is always TerminateRAZWI because the transaction is always terminated without an error response. No fault is reported in the SMMU.

B5.2.2 Downgrade types

The LTI response might require the transaction type to be downgraded when the translation gives permission for part of the operation but does not give permission for another part of the operation. [Table B5.3](#) shows the transaction type changes when a downgrade response is received.

Table B5.3: Response channel downgrade types

LATRANS	LRRESP	Translated transaction type	Effect
R-CMO	Downgrade1	R	The CMO part of the operation is not performed.
DCMO	Downgrade2	CMO	Change to the equivalent non-destructive CMO.

Continued on next page

Table B5.3 – Continued from previous page

LATRANS	LRRESP	Translated transaction type	Effect
R-DCMO	Downgrade1	R	The CMO part of the operation is not performed.
	Downgrade2	R-CMO	Change to the equivalent non-destructive R-CMO.
W-DCP	Downgrade1	W	If the final transaction attributes are not Snoopable Write-Back, the directed cache prefetch part of the operation is not performed. If the final transaction attributes are Snoopable Write-Back, the directed cache prefetch part of the operation is still performed but is forced to not specify a target cache.

B5.2.3 Restrictions based on LAFLOW

Table B5.4 shows the permitted encodings of **LRRESP** depend on the value of **LAFLOW** in the request.

Table B5.4: Response channel restrictions based on LAFLOW

LAFLOW	LRRESP encodings permitted
Stall	Success, Downgrade1, Downgrade2, FaultAbort, TerminateRAZWI
NoStall	Success, Downgrade1, Downgrade2, FaultAbort, TerminateRAZWI
PRI	Success, Downgrade1, Downgrade2, FaultAbort, TerminateRAZWI, FaultPRI
ATST	Success, Downgrade1, Downgrade2, FaultAbort, TerminateRAZWI

B5.2.4 Out of range address

When **LTI_MMU** is True and **LAADDR**[63:LTI_LRADDR_WIDTH] != 0 and any of the following conditions are true, **LRRESP** must be FaultAbort or TerminateRAZWI:

- **LAMMUV** is LOW
- **LAMMUV** is HIGH and **LAFLOW** = ATST and **LAIDENT** is HIGH

B5.3 Attribute restrictions for specific transaction types

Some transaction types require certain attributes. [Table B5.5](#) describes the restrictions that apply to **LRATTR** depending on the transaction type that is given by combining **LATRANS** and **LRRESP**, taking account of any downgrade required by **LRRESP**, where required.

Table B5.5: Response attribute restrictions for specific transaction types

LATRANS	Legal values of LRATTR when LTI_OUTER_CACHEABLE_MODE is	
	True	False
CMO, DCMO	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
	-	Non-snoopable Write-Back Allocate
R-CMO, R-DCMO, W-DCP	Snoopable Write-Back Non-allocate	Snoopable Write-Back Non-allocate
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
DCP	Snoopable Write-Back Non-allocate	Snoopable Write-Back Non-allocate
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
	Non-snoopable Write-Back Non-allocate	Non-snoopable Write-Back Non-allocate
	Non-snoopable Write-Back Allocate	Non-snoopable Write-Back Allocate
DHCMO	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
	Non-snoopable Write-Back Allocate	Non-snoopable Write-Back Allocate
SPEC	Device-nGnRnE	Device-nGnRnE
	Device-nGnRE	Device-nGnRE
	Device-nGRE	Device-nGRE
	Device-GRE	Device-GRE
	Normal Non-cacheable	Normal Non-cacheable
	-	Normal Inner Non-cacheable Outer Cacheable
	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate
	Non-snoopable Write-Back Allocate	Non-snoopable Write-Back Allocate

There is no attribute restriction for **LRATTR** when **LATRANS** is R, W, RW, W-CMO, or UNSPEC.

Note

In Issue A and Issue B, W-CMO had attribute restrictions on **LRATTR**. This is no longer required for further issues.

Chapter B6

Completion channel

This chapter defines the LTI completion (LC) channel. It contains the following sections:

- [B6.1 Signals](#)
- [B6.2 Completion channel characteristics](#)

B6.1 Signals

[Table B6.1](#) describes the signals in the LC channel.

Table B6.1: Completion channel signals

Signal	Category	Width	Description
LCVALID	Transport	1	Channel valid. When this signal is LOW, other TX signals on the LC channel are not valid.
LCCREDIT	Transport	1	Channel credit grant. LCCREDIT is an RX signal which flows in the other direction from other LC channel signals. It is not affected by LCVALID .
LCCTAG	Flow	LTI_CTAG_WIDTH	Translation completion tag. LCCTAG must match the value that is given in LRCTAG .
LCUSER	Impdef	LTI_LCUSER_WIDTH	IMPLEMENTATION DEFINED additional signaling.

B6.2 Completion channel characteristics

This section describes completions. Completions can be returned in any order.

B6.2.1 Deadlock avoidance

When a translation response is received by an LTI Manager, the completion must be returned without dependence on either:

- Subsequent software activity. However, when translation cache is used in the Manager, the completion is permitted to wait for software issued invalidation commands. This can trigger LT messages to be sent over LTI.
- Return of other translation responses where **LAFLOW** = Stall.

B6.2.2 Use of LTI translations for multiple transactions

Typically, each untranslated transaction causes an LTI request and response. However, it is possible to use a single LTI transaction for multiple untranslated transactions if the following conditions apply:

- The LTI response does not have **LRRESP** = FaultAbort, FaultPRI, or TerminateRAZWI.
- All Context and Transaction fields in the translation request are the same for all transactions.
- When **LTI_CACHE_SUPPORT** is True:
 - All data that is accessed by the transactions is within the same single address region of the size indicated by **LRSIZE**.
 - If the tag is permitted to be cached, the completion is not required to be returned on the LC channel in reasonable time.
 - If the tag is not permitted to be cached, the completion is required to be returned on the LC channel in reasonable time.
 - The completion must not be sent until all the transactions using the translation are complete and the translation is no longer retained in the Manager for subsequent transactions.
- When **LTI_CACHE_SUPPORT** is False:
 - All data that is accessed by the transactions is within the same single 4KB address region.
 - The completion is still returned on the LC channel in reasonable time.

The completion indicates that the Manager has finished using the translation. That means:

- Any downstream transactions using that translation are complete.
- The translation is no longer cached in the Manager.

Note

If the Manager wants to use a single LTI translation for multiple transactions, although it is permitted to do so without a translation cache, but it is recommended to use translation caches for better performance.

If the Manager uses **LAFLOW** = Stall, it is recommended that ordered requests always use TLBU for translation and never reuse a translation in the Manager.

All transactions using the translation must be globally observed before the completion message is sent by the LTI Manager.

CPU barrier instructions that must wait for completion of an invalidation might block while awaiting LC messages. The block may occur even if the translation used for that transaction has not been invalidated. If LC messages take a long time to return, it can impact CPU performance.

It is recommended that a single LTI response is only shared between transactions that can be issued together and are all outstanding at the same time, so that the overall latency to return the LC message is not substantially larger than the DRAM access latency. If there are requirements to decrease the LTI bandwidth, the translation caches can be used to prolong the LTI response reuse window to reduce redundant translations. Typically, it is recommended for the LTI Manager to rely on the translation caching of the LTI Subordinate and use a separate LTI request for each transaction, even if the LTI Manager knows that the translation is likely to be reused.

Chapter B7

Tag channel

This chapter defines the LTI tag (LT) channel. It contains the following section:

- [B7.1 Signals](#)

B7.1 Signals

The signals in the LT channel are described in [Table B7.1](#).

Table B7.1: Tag channel signals

Signal	Category	Width	Description
LTVALID	Transport	1	Channel valid. When this signal is LOW, other TX signals on the LT channel are not valid. This signal is present only when LTI_CACHE_SUPPORT is True.
LTCREDIT	Transport	1	Channel credit grant. This is an RX signal which flows in the other direction from other LT channel signals. It is not affected by LTVALID . This signal is present only when LTI_CACHE_SUPPORT is True.
LTCTAG	Flow	LTI_CTAG_WIDTH	Translation completion tag. This signal is present only when LTI_CACHE_SUPPORT is True.
LTUSER	Impdef	LTI_LTUSER_WIDTH	IMPLEMENTATION DEFINED additional signaling. This signal is present only when LTI_CACHE_SUPPORT is True.

B7.2 Tag message

A tag message is not part of the LTI transaction and does not have any relationship with the LTI request, response, or completion messages.

The tag message invalidates the translation caches in the Manager. The tag message indicates that only translations with CTAG value being equal to **LTCTAG** are permitted to be cached, and translations with all other CTAG values must not be cached. When a tag message is received, the Manager must invalidate all cache entries with non-matching CTAG values and stop caching new responses with non-matching CTAG values, and return the completions with non-matching CTAG values on the LC channel in reasonable time.

When the LTI Manager enters the OPEN state, no response is permitted to be cached until the first tag message is received.

Note

It is recommended that the TLBU sends an initial tag message with the same CTAG value that is found in the first response if it wants the first response to be cached by the Manager.

Two consecutive tag messages must not carry the same **LTCTAG** value while **LMOPENACK** is HIGH.

For the Manager, a window for CTAG value x is defined as follows:

- Begins when there is no outstanding LC messages with CTAG = x and a LR response with CTAG value x is received.
- Finishes after the last outstanding LC message with CTAG = x is sent.

During a CTAG window, if the Manager receives a LT message with a non-matching CTAG value while the CTAG is still permitted to be cached, there must not be a future LT message with a matching CTAG value in the same CTAG window.

The LT message must not wait for any outstanding completion on the LC channel.

The LT message must not wait for any outstanding response on the LR channel.

The LT message must not have dependence on the progress of any downstream transaction.

B7.2.1 Example invalidation sequences

In this section, the examples illustrate the invalidation sequence including the interactions between LTI and DTI.

When **LTI_CTAG_WIDTH** = 1, it is recommended that the TLBU sends the LT message when a **DTI_TBU_SYNC_REQ** message is received.

When **LTI_CTAG_WIDTH** > 1, it is recommended that the TLBU sends the LT message when a **DTI_TBU_INV_REQ** message is received.

In this section, x is the recent CTAG value which is permitted to be cached in the Manager before the LT message with CTAG = y . After the LT message with CTAG = y is received by the Manager, the CTAG value, x , is no longer permitted to be cached, and a new CTAG value, y , becomes permitted to be cached.

B7.2.1.1 Interface with **LTI_CTAG_WIDTH** = 1

Figure B7.1 shows an example of invalidation when **LTI_CTAG_WIDTH** = 1.

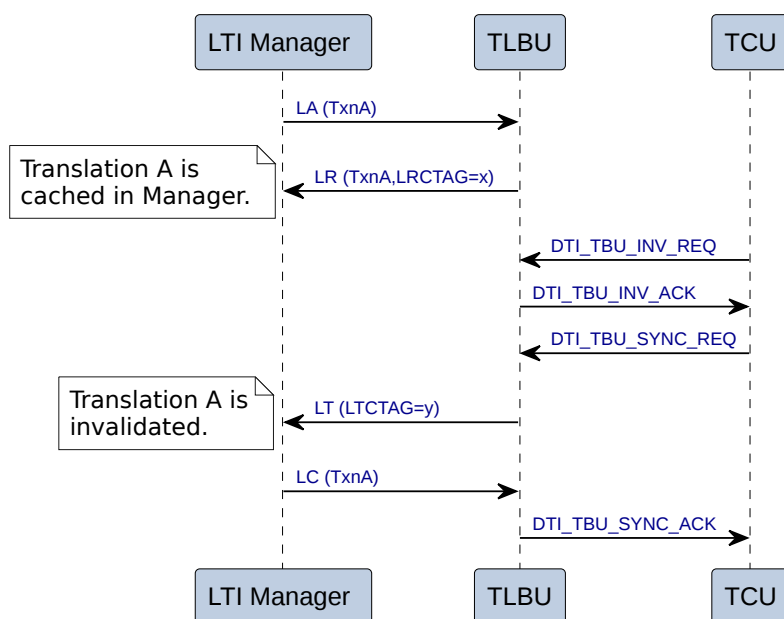


Figure B7.1: INV_ALL example

In Figure B7.1, Translation A with a CTAG value of x is firstly cached in the Manager and then invalidated by the LTI tag message with a different CTAG value of y . The LTI tag message is sent after TLBU receives the synchronization request from the TCU.

The LC for Transaction A is sent after Transaction A is invalidated from the cache and completed downstream. The TLBU waits for the LC before acknowledging the synchronization.

B7.2.1.2 Interface with LTI_CTAG_WIDTH > 1

Figure B7.2 shows an example of invalidation when LTI_CTAG_WIDTH > 1.

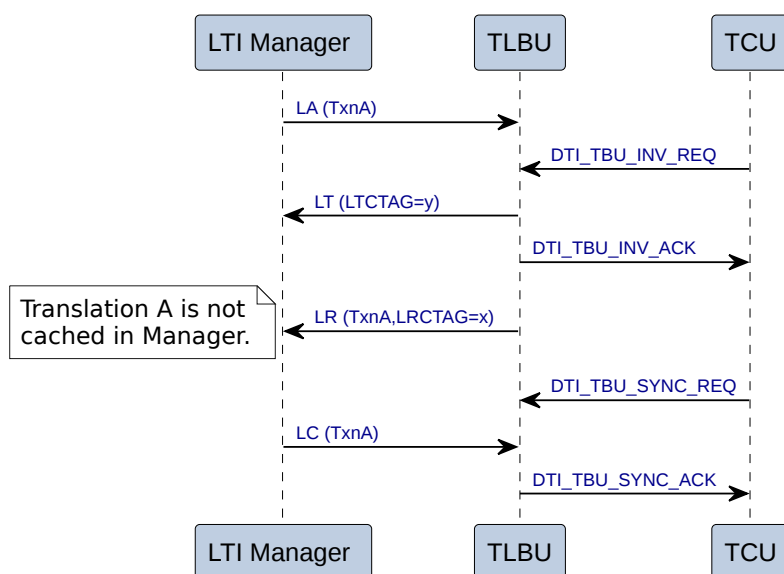


Figure B7.2: INV_CTAG example

In Figure B7.2, there are more than two CTAG values and the TLBU manages them more conductively. The TLBU

sends a LTI tag message with CTAG value of γ when it receives a DTI invalidation request. Translation A is invalidated in the TLBU and is assigned a different CTAG value of α . Therefore, Translation A is not cached in the Manager.

The LC for Transaction A is sent after Transaction A is invalidated from the cache and completed downstream. The TLBU waits for the LC before acknowledging the synchronization.

B7.3 Race between LT and LR channels

The LT and LR are two separate channels traveling to the same direction. The Manager and Subordinate might observe different order between LT and LR messages due to implementation choices, such as pipelining or clock domain crossing. If the LT message takes longer to travel across the link than the LR message, an LR message that is permitted to be cached on Subordinate interface might become not permitted to be cached on the Manager interface. It is recommended that the implementation considers minimizing such effects.

Chapter B8

Interface management

This chapter describes the LTI interface management function. It contains the following sections:

- [*B8.1 Interface management overview*](#)
- [*B8.2 Open and close handshake*](#)
- [*B8.3 Properties of interface states*](#)
- [*B8.4 Management signals*](#)

B8.1 Interface management overview

Signals with the LM prefix manage opening and closing the interface to enable power and clock control.

[Table B8.1](#) describes the interface management signals.

Table B8.1: Interface management signals

Signal	Driven by	Width	Description
LMOPENREQ	Manager	1	LTI interface open request
LMOPENACK	Subordinate	1	LTI interface open acknowledge
LMACTIVE	Manager	1	Request to open the interface or keep it open
LMASKCLOSE	Subordinate	1	Request to close the interface

B8.2 Open and close handshake

The **LMOPENREQ** and **LMOPENACK** signals control the LTI interface state. The LTI interface can be in one of the four states shown in [Table B8.2](#).

Table B8.2: LTI states

Interface state	LMOPENREQ	LMOPENACK
ST_CLOSED	0	0
ST_OPENING	1	0
ST_OPEN	1	1
ST_CLOSING	0	1

The LTI interface transitions through the states in the following order:

1. **LMOPENREQ** can transition from LOW to HIGH only if **LMOPENACK** is LOW.
2. **LMOPENREQ** can transition from HIGH to LOW only if **LMOPENACK** is HIGH.
3. **LMOPENACK** can transition from LOW to HIGH only if **LMOPENREQ** is HIGH.
4. **LMOPENACK** can transition from HIGH to LOW only if **LMOPENREQ** is LOW.

There must be no combinatorial paths between **LMOPENACK** and **LMOPENREQ** in either direction. A consequence of this is that the interface must spend at least one cycle in each state before moving to the next state.

Upon reset, the LTI interface is in state ST_CLOSED.

[Figure B8.1](#) shows how **LMOPENREQ** and **LMOPENACK** correspond to the interface states.

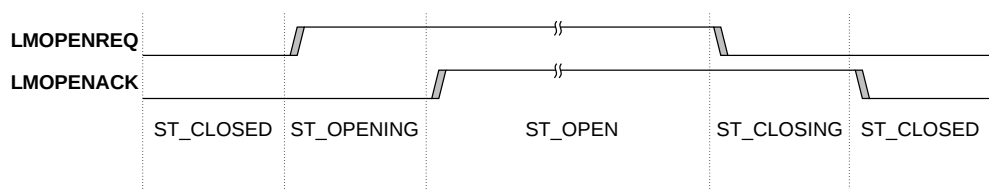


Figure B8.1: Interface states

B8.3 Properties of interface states

The Manager can only issue new requests on the LA channel when in the ST_OPEN state. **LAVALID** and **LCVALID** must be deasserted when **LMOPENREQ** is deasserted.

When the Manager enters the ST_OPEN state, cache contents are lost. The Manager must invalidate its caches before entering the ST_OPENING state.

The Manager must wait for all outstanding responses on the LR channel and send all outstanding completions on the LC channel before transitioning from the ST_OPEN state to the ST_CLOSING state. **LMOPENREQ** must be asserted while there are outstanding transactions on the interface. The Manager must not deassert **LMOPENREQ** until the cycle after it has asserted **LCVALID** for the completion.

LACREDIT and **LCCREDIT** can only be asserted in states ST_OPEN and ST_CLOSING. **LACREDIT** and **LCCREDIT** must be deasserted when **LMOPENACK** is deasserted.

LRCREDIT can only be asserted in the ST_OPEN state. **LRCREDIT** must be deasserted when **LMOPENREQ** or **LMOPENACK** are deasserted.

LTVALID can only be asserted when in the ST_OPEN and ST_CLOSING states. **LTVALID** must be deasserted when **LMOPENACK** is deasserted. It is possible for the Manager to receive LT messages when it is in the ST_CLOSING state. There must be no LT message presented to the Manager when the Manager is in the ST_CLOSED state.

LTCREDIT can only be asserted in the ST_OPEN state. **LTCREDIT** must be deasserted when **LMOPENREQ** or **LMOPENACK** are deasserted.

The Subordinate must wait for all outstanding completions on the LC channel before transitioning from ST_CLOSING to ST_CLOSED.

All credits on all channels are lost when in state ST_CLOSED. When the ST_OPEN state is entered, each channel TX has zero credits.

Note

It is recommended for an implementation to ensure that the Subordinate has received all outstanding completions before it transitions to the ST_CLOSING state.

It is recommended that the Subordinate does not wait for outstanding completions on the LC channel before asserting **LMASKCLOSE** to avoid adding long latency to the power down sequence, especially when the Manager includes translation caches.

B8.4 Management signals

This section describes the behavior of the interface management signals.

B8.4.1 LMASKCLOSE

The **LMASKCLOSE** signal enables the Subordinate to request that the interface is to be closed. For example, **LMASKCLOSE** may be asserted in response to a quiescence request on a Q-Channel interface on the Subordinate.

When **LMASKCLOSE** is asserted by the Subordinate and **LMACTIVE** is deasserted by the Manager, the Manager must, in a timely manner, either:

- Deassert **LMOPENREQ**, to move into **ST_CLOSING**.
- Assert **LMACTIVE**, to enable the Manager to issue new LTI requests. The Subordinate will usually then deassert **LMASKCLOSE**.

LMASKCLOSE must not be asserted when **LMOPENACK** is deasserted.

B8.4.2 LMACTIVE

The **LMACTIVE** signal serves two purposes:

- When asserted while the interface is closed, it indicates that the system should provide clock and power to the Subordinate and the interface can be opened.
- When asserted while the interface is opened, it indicates that the Subordinate should not request closing the interface.

There are no protocol rules linking **LMACTIVE** to outstanding LTI requests, however **LMACTIVE** is normally asserted while any LTI requests are outstanding.

LMACTIVE must be glitch-free and suitable for sampling in a different clock domain.

The Subordinate is permitted to remain in state **ST_OPENING** indefinitely when **LMACTIVE** is not asserted.

The Subordinate is permitted to assert **LMASKCLOSE** when **LMACTIVE** is asserted. This is because there might be a delay between **LMASKCLOSE** being asserted and it affecting **LMACTIVE**. However, it is expected that the Subordinate deasserts **LMASKCLOSE** when it detects that **LMACTIVE** is asserted.

The Manager is permitted to assert and deassert **LMACTIVE** without asserting **LMOPENREQ**. This is because **LMACTIVE** might be driven combinatorially, before **LMOPENREQ** can be asserted.

B8.4.2.1 Expected usage of LMACTIVE

A component can use an AXI **AWAKEUP** interface as a combinatorial input to **LMACTIVE**, because **AWAKEUP** is also required to be driven from a register and be glitch-free. It is expected that when **LMACTIVE** is asserted:

- If the Subordinate implements a Q-Channel, the Subordinate asserts **QACTIVE**.
- The Subordinate will deny Q-Channel quiescence requests.
- The Manager ignores **LMASKCLOSE**.

Examples of the use of **LMACTIVE** include:

- **LMASKCLOSE** might be asserted by the Subordinate when **LMACTIVE** is deasserted, but the new activity begins at the Manager before it begins to close the LTI interface. In this case, it asserts **LMACTIVE** and ignores **LMASKCLOSE**.
- The Manager might assert **LMACTIVE** to give early notice to the Subordinate to start its clocks, before the Manager is ready for the interface to be opened.

B8.4.2.2 Opening sequence example

Figure B8.2 shows an example of an interface opening sequence.

1. The Manager asserts **LMACTIVE** and **LMOPENREQ** to request opening the interface.
 - The Manager might assert **LMACTIVE** before **LMOPENREQ**.
 - It is permitted, but not expected, for the Manager to assert **LMACTIVE** after asserting **LMOPENACK**.
2. The Subordinate asserts **LMOPENACK** to accept opening the interface, and asserts **LACREDIT** and **LCCREDIT** to send request and completion credits. These can be asserted in any cycle that **LMOPENACK** is asserted.
3. The Manager sees that **LMOPENACK** is asserted and in the next cycle, asserts **LRCREDIT** to send response credits and starts providing LR credits. It also uses LA credits to start issuing requests on the LA channel.

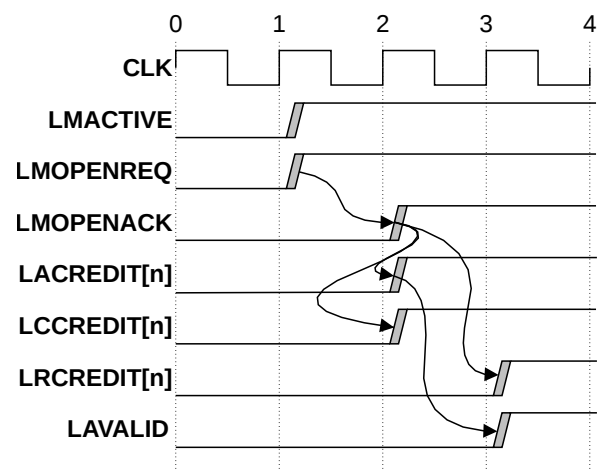


Figure B8.2: Example interface opening sequence

B8.4.2.3 Closing sequence example

Figure B8.3 shows an interface closing sequence that is initiated by the Subordinate.

1. The Subordinate observes **LMACTIVE** is not asserted and requests that the interface be closed by asserting **LMASKCLOSE**.
2. The Manager accepts the request to close the interface by deasserting **LMOPENREQ**.
3. The Subordinate completes the close sequence by deasserting **LMOPENACK** and **LMASKCLOSE**.

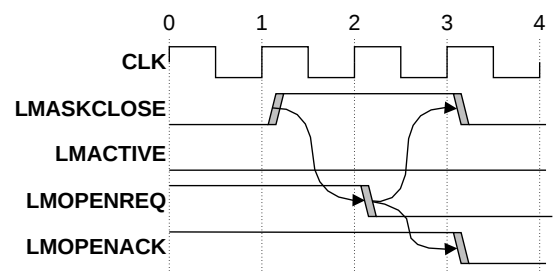


Figure B8.3: Closing sequence initiated by the Subordinate

Figure B8.4 shows a close request that is ignored by the Manager.

1. The Subordinate requests that the interface be closed by asserting **LMASKCLOSE**.
The Subordinate asserts **LMASKCLOSE** in the same cycle that the Manager asserts **LMACTIVE**, which indicates that the Manager has work to do, and will ignore **LMASKCLOSE**.
2. The Subordinate observes **LMACTIVE** and withdraws the interface closing request by deasserting **LMASKCLOSE**.

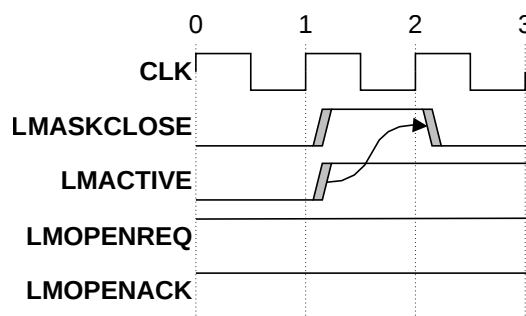


Figure B8.4: Denied interface closing sequence

The assertion of **LMACTIVE** and **LMASKCLOSE** are independent events, with no fixed relationship between them. Figure B8.4 shows them being asserted in the same cycle, but this is coincidental. A combinatorial path between the signals is not permitted.

Chapter B9

Clock and reset

This chapter describes the mapping strategy for the clock and reset. It contains the following section:

- [B9.1 Clock and reset](#)

B9.1 Clock and reset

An LTI interface is associated with a clock signal, **CLK** and a reset signal, **RESETn**.

[Table B9.1](#) shows the clock and reset signals.

Table B9.1: Clock and reset signals

Signal	Driven by	Width (bits)	Description
CLK	External	1	Global clock signal
RESETn	External	1	Global reset signal

Typically, an LTI interface shares its clock and reset with other interfaces on a component. For this reason, the clock and reset signals do not have an LTI-specific prefix in this specification.

An implementation might give the clock and reset signals different names from those signals used in the specification, or employ a different clock and reset strategy. It is recommended that there is a mapping between the clock and reset signals in an implementation and the signals that are defined here.

The signals that are not defined as asynchronous are sampled on the rising edge of **CLK**.

RESETn is asserted asynchronously and deasserted synchronously with **CLK**.

RESETn is active LOW.

- When **RESETn** is LOW, it is asserted and the other interface signals can be any value.
- When **RESETn** is HIGH, it is deasserted and the interface runs normally.

On the first rising edge of **CLK** where **RESETn** is HIGH, the following signals must be 0:

- **LxVALID**
- **LxCREDIT**
- **LMOPENREQ**
- **LMOPENACK**
- **LMASKCLOSE**

Chapter B10

Pipelining

This chapter defines pipelining requirements for LTI. It contains the following section:

- [*B10.1 Pipelining between Manager and Subordinate interfaces*](#)

B10.1 Pipelining between Manager and Subordinate interfaces

Pipeline stages or registers can be added to the signals between LTI Manager and Subordinate interfaces. The number of pipeline stages applied must comply with the following rules:

- All signals on each of the LA, LR, LC, and LT channels must be pipelined by the same number of cycles as all other signals in the same channel going in the same direction.
- **LMOPENREQ** must be pipelined by at least as many cycles as **LAVALID**, **LRCREDIT**, **LCVALID**, and **LTCREDIT**.
- **LMOPENACK** and **LMASKCLOSE** must each be pipelined by the same number of cycles, and both by at least as many cycles as **LACREDIT**, **LRVALID**, **LCCREDIT**, and **LTVALID**.

It is expected that in most implementations that require pipelining all LTI signals that go in the same direction are pipelined by the same number of cycles.

Chapter B11

Interface protection

This chapter specifies schemes for the protection of interfaces using parity signaling.

B11.1 Parity protection for interface signals

For safety-critical applications it is necessary to detect and possibly correct, transient and functional errors on individual wires within an SoC.

An error in a system component can propagate and cause multiple errors within connected components. Error detection and correction (EDC) is required to operate end-to-end, covering all logic and wires from source to destination.

One way to implement end-to-end protection, is to employ customized EDC schemes in components and implement a simple error detection scheme between components. Between these components there is no logic and single bit errors do not propagate to multi-bit errors. This section describes a parity scheme for detecting single-bit errors on the AMBA interface between components. Multi-bit errors can be detected if they occur in different parity signal groups. [Figure B11.1](#) shows locations where parity can be used in AMBA.

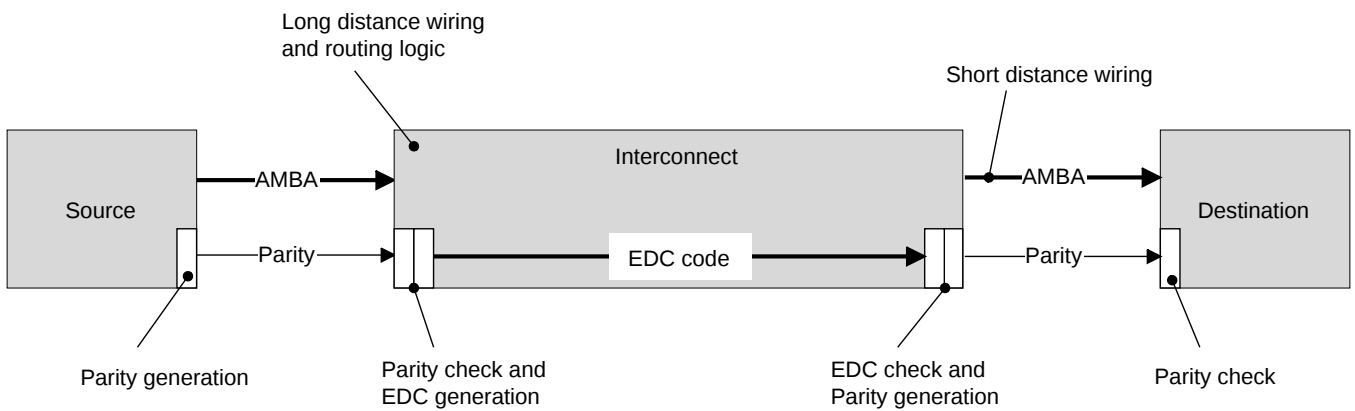


Figure B11.1: Parity use in AMBA

B11.1.1 Error detection behavior

This specification is not prescriptive regarding component or system behavior when a parity error is detected. Depending on the system and affected signals, a flipped bit can have a wide range of effects. It might be harmless, cause performance issues, data corruption, security violations, or deadlock. The transaction response is independent of parity error detection.

When an error is detected, the receiver can do any of the following:

- Terminate or propagate the transaction. Termination might or might not be protocol-compliant.
- Correct the parity check signal or propagate the signal in error.
- Update its memory or leave untouched. The location might be marked as poisoned.
- Signal an error response through other means, for example with an interrupt.

B11.1.2 Parity check signals

The parity check signals are listed in [Table B11.1](#). They have the following attributes and rules:

- Odd parity is used.
- Odd parity means that check signals are added to groups of signals on the interface and driven such that there is always an odd number of asserted bits in that group.
- Parity signals covering payload are defined such that in most cases, there are no more than 8 bits per group.

This limitation assumes that there is a maximum of 3 logic levels available in the timing budget for generating each parity bit.

- Parity signals covering critical control signals, which are likely to have a smaller timing budget available, are defined with a single odd parity bit. This single odd parity bit is the inversion of the original critical control signal.
- For a check signal that is wider than 1 bit:
 - Where a check signal covers multiple signals, parity is calculated by concatenating the signals in the order they are listed in [Table B11.1](#), with the first signal listed at the LSB.
 - Check bit [n] corresponds to bits [(8n+7):8n] in the payload.
 - If the payload is not an integer number of bytes, the most significant bit of the check signal covers fewer than 8-bits in the most significant portion of the payload.
- Check signals are synchronous to **CLK** and must be driven correctly in every cycle that the signal in the Check enable column is HIGH, see [Table B11.1](#).
- Parity signals must be driven appropriate to all the bits in the associated payload, irrespective of whether those bits are actively used in the transfer.
- If none of the signals covered by a check signal are present on an interface, then the check signal is omitted from the interface.

The following rules apply for **CHK** signals which cover multiple signals where one or more of the inputs or outputs are missing:

- If there is a missing signal output, the value is assumed to be the default for that signal.
- If there is an output signal with no corresponding input, the missing input cannot be assumed to take a fixed value. Therefore, the **CHK** signal cannot be used reliably.
- It is recommended that input signals that are part of a **CHK** group are either all present or all not present.

If a **CHK** signal misses any inputs or outputs, the default value is 0 for all the missing signals.

For more information about interface compatibility, see [Chapter B12 Interoperability](#).

Table B11.1: Parity check signals

Name	Signals covered	Width	Check enable
LAVALIDCHK	LAVALID	1	RESETn
LAVCCHK	LAVC	1	LAVALID
LACREDITCHK	LACREDIT	1	RESETn
LAIDCHK	LAID	$\text{ceil}(\text{LTI_ID_WIDTH}/8)$	LAVALID
LAOGCHK	LAOG	$\text{ceil}(\text{LTI_OG_WIDTH}/8)$	LAVALID
LASECSIDCHK	LASECSID	1	LAVALID
LASIDCHK	LASID	$\text{ceil}(\text{LTI_SID_WIDTH}/8)$	LAVALID
LASSIDCHK	LASSID LASSIDV	$\text{ceil}((\text{LTI_SSID_WIDTH} + 1)/8)$	LAVALID

Continued on next page

Table B11.1 – Continued from previous page

Name	Signals covered	Width	Check enable
LACTLCHK0	LAFLOW LAPRIV LAINST LAIDENT	1	LAVALID
LACTLCHK1	LAMMUV LAOGV	1	LAVALID
LACTLCHK2	LATRANS LAATTR	1	LAVALID
LAPMCHK	LAPM	1	LAVALID
LAPASUN- KNOWNCHK	LAPASUNKNOWN	1	LAVALID
LAPASCHK	LAPAS	1	LAVALID
LAADDRCHK	LAADDR	$\text{LTI_MMU} \times 8 :$ $\text{ceil}(\text{LTI_LRADDR_WIDTH}/8)$	LAVALID
LALOOPCHK	LALOOP	$\text{ceil}(\text{LTI_LOOP_WIDTH}/8)$	LAVALID
LATLBLOCCHK	LATLBLOC	$\text{ceil}(\text{LTI_TLBLOC_WIDTH}/8)$	LAVALID
LAUSERCHK	LAUSER	$\text{ceil}(\text{LTI_LAUSER_WIDTH}/8)$	LAVALID
LRVALIDCHK	LRVALID	1	RESETn
LRVCCHK	LRVC	1	LRVALID
LRCREDITCHK	LRCREDIT	1	RESETn
LRIDCHK	LRID	$\text{ceil}(\text{LTI_ID_WIDTH}/8)$	LRVALID
LRCTAGCHK	LRCTAG	$\text{ceil}(\text{LTI_CTAG_WIDTH}/8)$	LRVALID
LRRESPCHK	LRRESP	1	LAVALID
LRPASCHK	LRPAS	1	LAVALID
LRADDRCHK	LRADDR	$\text{ceil}(\text{LTI_LRADDR_WIDTH}/8)$	LRVALID
LRCTLCHK0	LRATTR LRHWATTR	1	LRVALID
LRMPAMCHK	LRMPAM	2	LRVALID
LRMECIDCHK	LRMECID	$\text{ceil}(\text{LTI_MECID_WIDTH}/8)$	LRVALID
LRSIZECHK	LRSIZE	1	LRVALID
LRLOOPCHK	LRLOOP	$\text{ceil}(\text{LTI_LOOP_WIDTH}/8)$	LRVALID
LRUSERCHK	LRUSER	$\text{ceil}(\text{LTI_LRUSER_WIDTH}/8)$	LRVALID
LCVALIDCHK	LCVALID	1	RESETn
LCCREDITCHK	LCCREDIT	1	RESETn
LCCTAGCHK	LCCTAG	$\text{ceil}(\text{LTI_CTAG_WIDTH}/8)$	LCVALID

Continued on next page

Table B11.1 – Continued from previous page

Name	Signals covered	Width	Check enable
LCUSERCHK	LCUSER	$\text{ceil}(\text{LTI_LCUSER_WIDTH}/8)$	LCVALID
LTVALIDCHK	LTVALID	1	RESETn
LTCREDITCHK	LTCREDIT	1	RESETn
LTCTAGCHK	LTCTAG	$\text{ceil}(\text{LTI_CTAG_WIDTH}/8)$	LTVALID
LTUSERCHK	LTUSER	$\text{ceil}(\text{LTI_LTUSER_WIDTH}/8)$	LTVALID
LMOPENREQCHK	LMOPENREQ	1	RESETn
LMOPENACKCHK	LMOPENACK	1	RESETn
LMACTIVECHK	LMACTIVE	1	RESETn
LMASKCLOSECHK	LMASKCLOSE	1	RESETn

Chapter B12

Interoperability

This appendix describes how to connect interfaces with different properties. It contains the following sections:

- [B12.1 *LTI-A compatibility*](#)
- [B12.2 *LTI_MMU and LTI_GPC properties*](#)
- [B12.3 *MPAM compatibility*](#)
- [B12.4 *LAMECID and LAHWATTR signals compatibility*](#)
- [B12.5 *LTI_CACHE_SUPPORT compatibility*](#)
- [B12.6 *LTI_GDI compatibility*](#)
- [B12.7 *LTI_PASUNKNOWN_PRESENT compatibility*](#)
- [B12.8 *LTI_OUTER_CACHEABLE_MODE compatibility*](#)
- [B12.9 *PAS signaling compatibility*](#)
- [B12.10 *PRIV and INST signaling compatibility*](#)

B12.1 LTI-A compatibility

[Table B12.1](#) shows the compatibility between an LTI-A interface and an LTI-D interface.

Table B12.1: Compatibility between an LTI-A interface and an LTI-D interface

	LTI-A Subordinate	LTI-D Subordinate
LTI-A Manager	Compatible	Compatible if LTI_MMU = True. Otherwise, not compatible. Subordinate LAMMUV input is tied HIGH. Subordinate LAIDENT input is tied LOW. Subordinate LAPASUNKNOWN input is tied LOW. Subordinate LRMECID output is unconnected. If LTI_GPC is True: – Subordinate LASECSID[1] input is tied LOW. – Subordinate LAPM input is tied LOW. – Subordinate LAPAS[2:1] input is tied 0. – Subordinate LRPAS[2:1] output is unconnected. – Subordinate LRMPAM.MPAM_SP[0] output is connected to Manager LRMPAM.MPAM_NS input. – Subordinate LRMPAM.MPAM_SP[1] output is unconnected. If LTI_GPC is False: – Subordinate LRMPAM.MPAM_NS output is connected to Manager LRMPAM.MPAM_NS input.
LTI-D Manager	Not compatible	See Table B12.2 .

B12.2 LTI_MMU and LTI_GPC properties

Table B12.2 shows the compatibility between a Manager and a Subordinate, according to the values of the LTI_MMU and LTI_GPC properties.

Table B12.2: Compatibility between LTI Manager and Subordinate interfaces

LTI Manager	LTI Subordinate		
	LTI_MMU = True LTI_GPC = False	LTI_MMU = True LTI_GPC = True	LTI_MMU = False LTI_GPC = True
LTI_MMU = True LTI_GPC = False	Compatible.	Compatible. Subordinate LASECSID [1] input is tied LOW. Subordinate LAPAS [2:1] input is tied 0. Subordinate LAPM is tied LOW. Subordinate LAPASUNKNOWN input is tied LOW. Subordinate LRPAS [2:1] output is unconnected. Subordinate LRMPAM.MPAM_SP [0] output is connected to Manager LRMPAM.MPAM_NS input. Subordinate LRMPAM.MPAM_SP [1] output is unconnected.	Not compatible
LTI_MMU = True LTI_GPC = True	Not compatible.	Compatible.	Not compatible.
LTI_MMU = False LTI_GPC = True	Not compatible.	Compatible. Subordinate LAADDR [63:LTI_LRADDR_WIDTH] input is tied to all 0. Subordinate LAFLOW input is tied to any value. Subordinate LASECSID input is tied to any value. Subordinate LASID input is tied to any value. Subordinate LASSIDV input is tied to any value. Subordinate LASSID input is tied to any value. Subordinate LAPRIV input is tied to any value. Subordinate LAINST input is tied to any value. Subordinate LAPM input is tied to any value. Subordinate LAPASUNKNOWN input is tied to any value. Subordinate LAIDENT is tied to any value.	Compatible.

B12.3 MPAM compatibility

The compatibility of **LRMPAM** between the Manager and Subordinate is affected by properties **LTI_MPAM_SUPPORT** and **LTI_GPC**.

When connecting a LTI Manager with **LTI_MPAM_SUPPORT** = **MPAM_9_1** to a LTI Subordinate with **LTI_MPAM_SUPPORT** = **MPAM_12_1**:

- Subordinate **LRMPAM**.PARTID[8:0] output is connected to Manager **LRMPAM**.PARTID input and Subordinate **LRMPAM**.PARTID[11:9] output is unconnected.
- Subordinate **LRMPAM**.PMG output is connected to Manager **LRMPAM**.PMG input.
- The connection of the PARTID space field **LRMPAM**.MPAM_SP or **LRMPAM**.MPAM_NS is also affected by **LTI_GPC** property. See [Table B12.2](#) for more details.

When connecting a LTI Manager with **LTI_MPAM_SUPPORT** = **MPAM_12_1** to a LTI Subordinate with **LTI_MPAM_SUPPORT** = **MPAM_9_1**:

- Subordinate **LRMPAM**.PARTID output is connected to Manager **LRMPAM**.PARTID[8:0] input and Manager **LRMPAM**.PARTID[11:9] input is tied to 0.
- Subordinate **LRMPAM**.PMG output is connected to Manager **LRMPAM**.PMG input.
- The connection of the PARTID space field **LRMPAM**.MPAM_SP or **LRMPAM**.MPAM_NS is also affected by **LTI_GPC** property. See [Table B12.2](#) for more details.

When connecting a LTI Manager with **LTI_MPAM_SUPPORT** != False to a LTI Subordinate with **LTI_MPAM_SUPPORT** = False, the system must add MPAM information. The default is IMPLEMENTATION DEFINED, but one option is:

- Manager **LRMPAM**.PARTID and **LRMPAM**.PMG inputs are tied to 0.
- When Manager **LTI_GPC** is True, copy the physical address space in **LRPAS** onto Manager **LRMPAM**.MPAM_SP input.
- When Manager **LTI_GPC** is False, copy the physical address space **LRPAS** onto Manager **LRMPAM**.MPAM_NS input.

When connecting a LTI Manager with **LTI_MPAM_SUPPORT** = False to a LTI Subordinate with **LTI_MPAM_SUPPORT** != False, Subordinate **LRMPAM** output is unconnected.

B12.4 LAMECID and LAHWATTR signals compatibility

Table B12.3 shows the compatibility of **LAMECID** and **LAHWATTR** signals between LTI-B and LTI-D interfaces.

Table B12.3: Compatibility of LAMECID and LAHWATTR signals

	LTI-B Subordinate	LTI-D Subordinate
LTI-B Manager	Compatible.	Compatible. Manager must drive LAMECID and LAHWATTR outputs to 0. Manager LAMECID and LAHWATTR outputs unconnected.
LTI-D Manager	Compatible. Subordinate LAMECID and LAHWATTR inputs are tied to 0.	Compatible.

Note

The **LAMECID** and **LAHWATTR** signals are only defined on the LTI-B interface. These signals are not described in this issue of the specification. For more details, see Issue B.

B12.5 LTI_CACHE_SUPPORT compatibility

Table B12.4 shows the compatibility between Manager and Subordinate interfaces according to the LTI_CACHE_SUPPORT property.

Table B12.4: LTI_CACHE_SUPPORT property compatibility

	Subordinate:True	Subordinate:False
Manager:True	Compatible.	Not compatible.
Manager:False	Not compatible.	Compatible.

B12.6 LTI_GDI compatibility

Table B12.5 shows the compatibility between Manager and Subordinate interfaces according to the LTI_GDI property.

Table B12.5: LTI_GDI property compatibility

	Subordinate:True	Subordinate:False
Manager:True	Compatible.	Not compatible.
Manager:False	Compatible. Subordinate LAPAS[2] input is tied to 0. Subordinate LAPM input is tied to 0. Subordinate LRPAS[2] output is unconnected.	Compatible.

B12.7 LTI_PASUNKNOWN_PRESENT compatibility

Table B12.6 shows the compatibility between Manager and Subordinate interfaces according to the LTI_PASUNKNOWN_PRESENT property.

Table B12.6: LTI_PASUNKNOWN_PRESENT property compatibility

	Subordinate:True	Subordinate:False
Manager:True	Compatible.	Not compatible.
Manager:False	Compatible. Subordinate LAPASUNKNOWN input is tied LOW.	Compatible.

B12.8 LTI_OUTER_CACHEABLE_MODE compatibiltiy

[Table B12.7](#) shows the compatibility between Manager and Subordinate interfaces according to the LTI_OUTER_CACHEABLE_MODE property.

Table B12.7: LTI_OUTER_CACHEABLE_MODE property compatibility

	Subordinate:True	Subordinate:False
Manager:True	Compatible.	Not compatible.
Manager:False	Not compatible.	Compatible.

B12.9 PAS signaling compatibility

When connecting to an LTI-D interface to an interface of LTI-C or earlier:

- **LAPAS[0]** is connected to **LAPROT[1]**.
- **LAPAS[1]** is connected to **LANSE**.
- **LRPAS[0]** is connected to **LRPROT[1]**.
- **LRPAS[1]** is connected to **LRNSE**.

B12.10 PRIV and INST signaling compatibility

When connecting an LTI-D interface to an LTI-C interface or earlier:

- **LAPRIV** is connected to **LAPROT[0]**.
- **LAINST** is connected to **LAPROT[2]**.
- When connecting a Manager with LTI-C or earlier to a Subordinate with LTI-D:
 - If **LATRANS** == SPEC, Manager input **LRPROT[0]** is tied to 0.
 - If **LATRANS** != SPEC, Manager input **LRPROT[0]** is tied to 1.
 - Manager input **LRPROT[2]** is tied to 0.
- When connecting a Manager with LTI-D to a Subordinate with LTI-C or earlier:
 - Subordinate outputs **LRPROT[0]** and **LRPROT[2]** are not connected.

Chapter B13

Pseudocode

This chapter provides example implementations of the requirements specified in this document.

The pseudocode language is as described in the *Arm® Architecture Reference Manual for A-profile architecture*.

It contains the following sections:

- [B13.1 *PermissionCheckDHCMO*](#)

B13.1 PermissionCheckDHCMO

This section describes the permission check required for the DHCMO transaction.

```
// PermissionCheckDHCMO()
// =====
// Check DRE and RWX permissions for a DHCMO transaction.
boolean PermissionCheckDHCMO(bit lammuv,
                             bits(2) lasecsid,
                             bits(2) laflow,
                             bit lapriv,
                             bits(3) lrpas,
                             DTI_TBU_TRANS_RESP resp)

effective_inst = (resp.INSTCFG == "Instruction");
effective_priv = ((resp.PRIVCFG == "Use incoming") && lapriv) ||
                 (resp.PRIVCFG == "Privileged");

allow_w = effective_priv ? resp.ALLOW_PW : resp.ALLOW_UW;

if resp.BYPASS then
    if (resp.BP_TYPE == DPTBypass) ||
        (resp.BP_TYPE == StreamBypass && laflow == ATST) then
        dre = resp.DRE;
    else
        dre = '1';

    allow_r = '1';
    allow_x = (lrpas != Non-secure) ||
              (lasecsid == Non-secure) ||
              (lasecsid == Secure && resp.ALLOW_NSX);
else
    dre = resp.DRE;
    allow_r = effective_priv ? resp.ALLOW_PR : resp.ALLOW_UR;
    allow_x = effective_priv ? resp.ALLOW_PX : resp.ALLOW_UX;

return (!lammuv ||
        ((effective_inst || allow_r) && (!effective_inst || allow_x) &&
         allow_w &&
         dre));
```

Part C

Appendices

Chapter C1

Considerations for AXI5

This appendix describes how to map LTI concepts onto an AXI5 interface. It contains the following sections:

- *C1.1 LATRANS mapping*
- *C1.2 LAATTR mapping*
- *C1.3 LAIDENT mapping*
- *C1.4 LRATTR mapping*
- *C1.5 LRMECID mapping*
- *C1.6 LRHWATTR mapping*
- *C1.7 xRESP mapping*
- *C1.8 PAS mapping*
- *C1.9 PRIV and INST mapping*
- *C1.10 Transactions that are legal in AXI5 and illegal in LTI*
- *C1.11 Memory tagging*
- *C1.12 AXI opcode modifications*

C1.1 LATRANS mapping

Table C1.1 shows the expected mapping of AXI5 transactions to LATRANS.

Table C1.1: LATRANS mapping

AXI5 AR channel transactions	AXI5 AW channel transactions	LATRANS
-	StashTransaction	SPEC
ReadNoSnoop ReadOnce	-	R
-	WriteNoSnoop, WriteUniquePtl, WriteUniqueFull, WriteNoSnoopFull, WriteDeferrable, WriteZero, WriteUniquePtlStash and WriteUniqueFullStash with AWSTASHNIDEN = 0 and AWSTASHLPIDEN = 0	W
-	AtomicLoad, AtomicSwap, AtomicCompare, AtomicStore ^a	RW
CleanShared CleanInvalid CleanSharedPersist	CMO	CMO
ReadOnceCleanInvalid	-	R-CMO
-	WritePtlCMO, WriteFullCMO	W-CMO
MakeInvalid	-	DCMO
ReadOnceMakeInvalid	-	R-DCMO
-	InvalidateHint	DHCMO
-	StashOnceShared, StashOnceUnique	DCP
-	WriteUniquePtlStash and WriteUniqueFullStash with AWSTASHNIDEN = 1 or AWSTASHLPIDEN = 1	W-DCP
-	UnstashTranslation	UNSPEC

^a AtomicStore operations are defined as type RW, not W, even though no read data is returned. This is required by the SMMUv3 architecture.

The following AXI5 transactions are not supported for mapping to LTI:

- ReadShared
- ReadClean
- WriteBackFull
- WriteEvictFull
- Prefetch
- DVM Complete

C1.2 LAATTR mapping

In a coherent memory system, all coherent Managers must use consistent attributes to the same memory location. The mapping of Armv8 memory types to AMBA memory types must be done consistently so this property is not broken.

[Table C1.2](#) shows the recommended mapping of the AXI5 **AxCACHE** and **AxDOMAIN** signals to **LAATTR** values.

Table C1.2: AxCACHE and AxDOMAIN mapping to LAATTR

AxCACHE	AxDOMAIN	LAATTR
Device Non-bufferable	System	Device-nGnRnE
Device Bufferable	System	Device-nGnRE
Normal Non-cacheable Bufferable Normal Non-cacheable Non-bufferable	System, Non-shareable, Shareable	Normal Non-cacheable
Write-Through No-allocate ^a Write-Through Read and Write-Allocate ^b	Non-shareable, Shareable	Normal Non-cacheable
Write-Back No-allocate ^a	Shareable	Snoopable Write-Back No-allocate
Write-Back Read and Write-Allocate ^b	Shareable	Snoopable Write-Back Allocate
Write-Back No-allocate ^a	Non-shareable	Non-snoopable Write-Back No-allocate
Write-Back Read and Write-Allocate ^b	Non-shareable	Non-snoopable Write-Back Allocate

^a Includes Read-Allocate for writes, and Write-Allocate for reads.

^b Includes Read-Allocate for reads, and Write-Allocate for writes.

When **LATRANS** is DCP, W-DCP, R-CMO, or R-DCMO, the allocation hint in **LAATTR** is Allocate if the memory type in **AxCACHE** is Normal Non-cacheable or Write-Through.

C1.3 LAIDENT mapping

An AXI5 interface does not have an equivalent signal to **LAIDENT**.

When mapping AXI5 transactions to LTI, **LAIDENT** is always 0.

C1.4 LRATTR mapping

In a coherent memory system, all coherent Managers must use consistent attributes to the same memory location. The mapping of Armv8 memory types to AMBA memory types must be done consistently so this property is not broken.

Table C1.3 shows the recommended mapping of the **LRATTR** values to the AXI5 **AxCACHE** and **AxDOMAIN** signals.

Table C1.3: LRATTR mapping to AxCACHE and AxDOMAIN

LRATTR	AxCACHE	AxDOMAIN
Device-nGnRnE	Device Non-bufferable	System
Device-nGnRE	Device Bufferable	System
Device-nGRE		
Device-GRE		
Normal Non-cacheable	Normal Non-cacheable Bufferable	System
Normal Inner Non-cacheable Outer Cacheable ^a		
Snoopable Write-Back No-allocate	Write-Back No-allocate	Shareable
Snoopable Write-Back Allocate	Write-Back Read and Write-Allocate	Shareable
Non-snoopable Write-Back No-allocate	Write-Back No-allocate	Non-shareable
Non-snoopable Write-Back Allocate	Write-Back Read and Write-Allocate	Non-shareable

^a Normal Inner Non-cacheable Outer Cacheable is only possible when LTI_OUTER_CACHEABLE_MODE is False.

It is recommended that transactions with **AxBURST** = **FIXED** are terminated with a **SLVERR** response. It is recommended that Managers which are translated by an SMMU do not use the **FIXED** burst type.

If **AxLOCK** = 1, **AxCACHE** is not permitted to indicate a Cacheable type without additional knowledge of the structure of the downstream interconnect. If an AXI5 transaction with **AxLOCK** = 1 translates to any Normal Write-Back memory type, it is recommended that it be output with **AxLOCK** = 0. It is recommended that Managers requiring semaphore-type operations use the AXI5 atomic transactions.

If **AWSNNOOP** = **WriteDeferrable**, **AWDOMAIN** is required to be **SYS**. If the memory type in **LRATTR** is Normal Write-Back, it is recommended that the Manager terminates the transaction with a **SLVERR** response.

When **WriteUniquePtlStash** or **WriteUniqueFullStash** transaction has final transaction attributes **Snoopable Write-Back**, but **LRRESP** = **Downgrade1**, output **AWSTASHNIDEN** = 0 and **AWSTASHLPIDEN** = 0.

C1.5 LRMECID mapping

It is recommended to map **LRMECID** as follows:

- When **LAMMUV** is 1 and **LRPAS** is not NSP, the output **AxMECID** is **LRMECID**.
- When **LAMMUV** is 0 or **LRPAS** is NSP, the output **AxMECID** is the supplied MECID or 0 if there is no supplied MECID. It is IMPLEMENTATION DEFINED how the MECID is supplied.

C1.6 LRHWATTR mapping

It is recommended to map **LRHWATTR** as follows:

- When **LAMMUV** is 1, the output **AxPBHA** is **LRHWATTR**.
- When **LAMMUV** is 0, the output **AxPBHA** is the supplied PBHA or 0 if there is no supplied PBHA. It is IMPLEMENTATION DEFINED how the PBHA is supplied.

C1.7 xRESP mapping

Table C1.4 shows the relative mapping of **LRRESP** to AXI5 **RRESP** and **BRESP** signals when the transaction is terminated by the LTI Manager.

Table C1.4: LRRESP to xRESP mapping

LRRESP	xRESP
Success, Downgrade1, Downgrade2	If (LATRANS = SPEC or UNSPEC), xRESP is OKAY. Otherwise, xRESP is propagated from the downstream transaction.
FaultAbort	SLVERR
TerminateRAZWI	OKAY
FaultPRI	TRANSFAULT

C1.8 PAS mapping

It is recommended to map **LAPAS** as follows:

- Either **AxPROT[1]** or **AxPAS[0]** is **LAPAS[0]**.
- Either **AxNSE** or **AxPAS[1]** is **LAPAS[1]**.
- **AxPAS[2]** is **LAPAS[2]**.

It is recommended to map **LRPAS** as follows:

- Either **AxPROT[1]** or **AxPAS[0]** is **LRPAS[0]**.
- Either **AxNSE** or **AxPAS[1]** is **LRPAS[1]**.
- **AxPAS[2]** is **LRPAS[2]**.

C1.9 PRIV and INST mapping

It is recommended to map input **AxPRIV/AxPROT[0]** and **AxINST/AxPROT[2]** as follows:

- Either **AxPRIV** or **AxPROT[0]** is **LAPRIV**.
- Either **AxINST** or **AxPROT[2]** is **LAINST**.

It is recommended to map output **AxPRIV/AxPROT[0]** and **AxINST/AxPROT[2]** as follows:

- Either **AxPRIV** or **AxPROT[0]** is always 1 (Privileged access).
- Either **AxINST** or **AxPROT[2]** is always 0 (Data access).

C1.10 Transactions that are legal in AXI5 and illegal in LTI

Certain transactions are legal in AXI5 but illegal in LTI. The following modifications are required to be made to transactions so that they are legal in LTI:

- Write transactions marked as Instruction (**AWPROT**[2] = 1) are converted to Data (**AWPROT**[2] = 0).
- ATST transactions (**AxMMUFLOW** = ATST and **AxMMUSSIDV** = 0) marked as Instruction (**AxPROT**[2] = 1) are converted to Data (**AxPROT**[2] = 0).
- ATST transactions (**AxMMUFLOW** = ATST and **AxMMUSSIDV** = 0) marked as Privileged (**AxPROT**[0] = 1) are converted to Unprivileged (**AxPROT**[0] = 0).

C1.11 Memory tagging

The SMMUv3 architecture does not support MTE. If Memory Tagging is supported, it must not be used by transactions requiring translation, that is, when **LAMMUV** is HIGH.

When a transaction is received on the AR channel with **ARMMUVALID** HIGH, **ARTAGOP** must be 0b00, Invalid.

When a transaction is received on the AW channel with **AWMMUVALID** HIGH:

- **AWTAGOP** must be 0b00, Invalid.
- **WTAG** must be 0.
- **WTAGUPDATE** must be 0.

C1.12 AXI opcode modifications

An opcode modification can happen if **LRRESP** is a downgrade type which changes the opcode, or the translated attributes are not supported by the opcode.

Table C1.5 shows the AXI opcode modifications.

Table C1.5: AXI opcode modifications

Input opcode	Output opcode	Conditions
ReadNoSnoop	ReadOnce	Final memory type is Snoopable Write-Back.
ReadOnce	ReadNoSnoop	Final memory type is not Snoopable Write-Back.
ReadOnceCleanInvalid	ReadNoSnoop	LRRESP is Downgrade1.
ReadOnceMakeInvalid	ReadNoSnoop	LRRESP is Downgrade1.
ReadOnceMakeInvalid	ReadOnceCleanInvalid	LRRESP is Downgrade2.
MakeInvalid	CleanInvalid	LRRESP is Downgrade2.
WriteNoSnoop	WriteUniquePtl	Final memory type is Snoopable Write-Back.
WriteNoSnoopFull	WriteUniqueFull	Final memory type is Snoopable Write-Back.
WriteNoSnoopFull	WriteNoSnoop	Final memory type is not Normal Write-Back.
WriteUniquePtl	WriteNoSnoop	Final memory type is not Snoopable Write-Back.
WriteUniqueFull	WriteNoSnoop or WriteNoSnoopFull	Final memory type is not Snoopable Write-Back.
WritePtlCMO	WriteNoSnoop	Final memory type is not Normal Write-Back.
WriteFullCMO	WriteNoSnoop or WriteNoSnoopFull	Final memory type is not Normal Write-Back.
WriteUniquePtlStash	WriteNoSnoop	Final memory type is not Snoopable Write-Back.
WriteUniqueFullStash	WriteNoSnoop or WriteNoSnoopFull	Final memory type is not Snoopable Write-Back.

Chapter C2

Considerations for DTI

This appendix describes how to map LTI concepts onto DTI-TBU. It contains the following sections:

- [*C2.1 DTI_TBU_TRANS_REQ.PERM mapping*](#)
- [*C2.2 Special handling for specific LATRANS values*](#)
- [*C2.3 Attribute mapping*](#)
- [*C2.4 DTI_TBU_TRANS_FAULT.FAULT_TYPE mapping*](#)

C2.1 DTI_TBU_TRANS_REQ.PERM mapping

Table C2.1 shows how the value of **LATRANS** in an LTI request is expected to map to a value of **DTI_TBU_TRANS_REQ.PERM** in a DTI request.

Table C2.1: DTI_TBU_TRANS_REQ.PERM mapping

LATRANS	DTI_TBU_TRANS_REQ.PERM
SPEC, DCP, DHCMO	SPEC
R, CMO, R-CMO, DCMO, R-DCMO	R
W, W-DCP	W
RW, W-CMO	RW

C2.2 Special handling for specific LATRANS values

Some transaction types require special handling when calculating permissions and attributes.

C2.2.1 LATRANS = DCP

A cache stashing operation is only meaningful for Cacheable memory locations. If the final transaction attributes are not Normal Write-Back, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.

The allocate hint does not affect **LRRESP**.

If DCP permission is not granted, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.

If neither R, W, or X permissions are granted at the appropriate privilege level, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.

C2.2.2 LATRANS = W-DCP

A cache stashing operation is only meaningful for Cacheable and coherent memory locations. If the final transaction attributes are not Snoopable Write-Back, convert **LRRESP** = Success into **LRRESP** = Downgrade1.

If DCP permission is not granted, convert **LRRESP** = Success into **LRRESP** = Downgrade1.

C2.2.3 LATRANS = CMO

Cache Maintenance Operations do not have a memory type.

When LTI_OUTER_CACHEABLE_MODE is True, return **LRATTR** = Snoopable Write-Back Allocate.

When LTI_OUTER_CACHEABLE_MODE is False, if the calculated shareability is Non-shareable, return **LRATTR** = Non-snoopable Write-Back Allocate, otherwise return **LRATTR** = Snoopable Write-Back Allocate.

C2.2.4 LATRANS = R-CMO

Deallocating reads are intended to operate on Cacheable and coherent memory locations. If the final transaction attributes are not Snoopable Write-Back, convert **LRRESP** = Success into **LRRESP** = Downgrade1.

The allocate hint does not affect **LRRESP**.

C2.2.5 LATRANS = DCMO

Cache Maintenance Operations do not have a memory type.

When LTI_OUTER_CACHEABLE_MODE is True, return **LRATTR** = Snoopable Write-Back Allocate.

When LTI_OUTER_CACHEABLE_MODE is False, if the calculated shareability is Non-shareable, return **LRATTR** = Non-snoopable Write-Back Allocate, otherwise return **LRATTR** = Snoopable Write-Back Allocate.

If either W permission is not granted at the appropriate privilege level or DRE permission is not granted, convert **LRRESP** = Success into **LRRESP** = Downgrade2.

C2.2.6 LATRANS = R-DCMO

Deallocating reads are intended to operate on Cacheable and coherent memory locations. If the final transaction attributes are not Snoopable Write-Back, convert **LRRESP** = Success into **LRRESP** = Downgrade1.

The allocate hint does not affect **LRRESP**.

Otherwise, if either W permission is not granted at the appropriate privilege level or DRE permission is not granted, convert **LRRESP** = Success into **LRRESP** = Downgrade2.

C2.2.7 LATRANS = DHCMO

A destructive hint operation is only meaningful for Cacheable memory locations. If the final transaction attributes are not Normal Write-Back, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.

More specifically:

- When LTI_OUTER_CACHEABLE_MODE is True:
 - If the calculated memory type is Normal-iNC-oWB, return **LRATTR** = Non-snoopable Write-Back Allocate.
 - If the calculated memory type is Normal-iWB-oWB, return **LRATTR** = Snoopable Write-Back Allocate.
 - If the calculated memory type is not Normal-iNC-oWB or Normal-iWB-oWB, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.
- When LTI_OUTER_CACHEABLE_MODE is False:
 - If the calculated memory type is Normal-iWB-oWB and the calculated shareability is Non-shareable, return **LRATTR** = Non-snoopable Write-Back Allocate.
 - If the calculated memory type is Normal-iWB-oWB and the calculated shareability is Shareable, return **LRATTR** = Snoopable Write-Back Allocate.
 - If the calculated memory type is not Normal-iWB-oWB, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI.

If the following expression is False, convert **LRRESP** = Success into **LRRESP** = TerminateRAZWI:

```
PermissionCheckDHCMO (LAMMUV,
                      LASECSID,
                      LAFLOW,
                      LAPRIV,
                      LRPAS,
                      DTI_TBU_TRANS_RESP)
```

In this specification, the term ‘DTI_TBU_TRANS_RESP’ represents both the DTI_TBU_TRANS_RESP and DTI_TBU_TRANS_RESPEX messages.

C2.3 Attribute mapping

This section describes the mapping between LTI attributes and Armv8 attributes used by DTI.

[Table C2.2](#) shows the meanings of the abbreviations used in this section.

Table C2.2: Attribute abbreviations

Abbreviation	Meaning
iNC	inner Non-cacheable
iWT	inner Write-Through
iWB	inner Write-Back
oNC	outer Non-cacheable
oWT	outer Write-Through
oWB	outer Write-Back attributes

C2.3.1 LTI to Armv8 conversion

For meaningful cases, the Armv8 type is Read Allocate, Write Allocate, and Non-transient for the Inner cacheability domain. The allocation hints for the Armv8 type are shown in [Table C2.3](#) and [Table C2.4](#).

[Table C2.3](#) shows the LTI to Armv8 conversion values when LTI_OUTER_CACHEABLE_MODE is True.

Table C2.3: LTI to Armv8 conversion when LTI_OUTER_CACHEABLE_MODE is True

LTI attribute	Armv8 type	Armv8 shareability
Device-nGnRnE	Device-nGnRnE	Outer Shareable
Device-nGnRE	Device-nGnRE	Outer Shareable
Device-nGRE	Device-nGRE	Outer Shareable
Device-GRE	Device-GRE	Outer Shareable
Normal Non-cacheable	Normal-iNC-oNC	Outer Shareable
Snoopable Write-Back No-allocate	Normal-iWB-oWB Read No-allocate Write No-allocate	Outer Shareable
Snoopable Write-Back Allocate	Normal-iWB-oWB Read-Allocate Write-Allocate	Outer Shareable
Non-snoopable Write-Back No-allocate	Normal-iNC-oWB Read No-allocate Write No-allocate	Outer Shareable
Non-snoopable Write-Back Allocate	Normal-iNC-oWB Read-Allocate Write-Allocate	Outer Shareable

[Table C2.4](#) shows the LTI to Armv8 conversion values when LTI_OUTER_CACHEABLE_MODE is False.

Table C2.4: LTI to Armv8 conversion when LTI_OUTER_CACHEABLE_MODE is False

LTI attribute	Armv8 type	Armv8 shareability
Device-nGnRnE	Device-nGnRnE	Outer Shareable
Device-nGnRE	Device-nGnRE	Outer Shareable
Device-nGRE	Device-nGRE	Outer Shareable
Device-GRE	Device-GRE	Outer Shareable
Normal Non-cacheable	Normal-iNC-oNC	Outer Shareable
Normal Inner Non-cacheable Outer cacheable		
Snooperable Write-Back No-allocate	Normal-iWB-oWB Read No-allocate Write No-allocate	Outer Shareable
Snooperable Write-Back Allocate	Normal-iWB-oWB Read-Allocate Write-Allocate	Outer Shareable
Non-snooperable Write-Back No-allocate	Normal-iWB-oWB Read No-allocate Write No-allocate	Non-shareable
Non-snooperable Write-Back Allocate	Normal-iWB-oWB Read-Allocate Write-Allocate	Non-shareable

C2.3.2 Armv8 to LTI conversion

Table C2.5: Armv8 to LTI conversion

Armv8 type	Armv8 shareability	LTI attribute when LTI_OUTER_CACHEABLE_MODE is	
		True	False
Device-nGnRnE	Outer Shareable	Device-nGnRnE	Device-nGnRnE
Device-nGnRE	Outer Shareable	Device-nGnRE	Device-nGnRE
Device-nGRE	Outer Shareable	Device-nGRE	Device-nGRE
Device-GRE	Outer Shareable	Device-GRE	Device-GRE
Normal-iNC-oNC Normal-iWT-oNC Normal-iWB-oNC	Non-shareable, Outer Shareable, Inner Shareable	Normal Non-cacheable	Normal Non-cacheable
Normal-iNC-oWT Normal-iWT-oWT Normal-iWB-oWT Normal-iWT-oWB	Non-shareable, Outer Shareable, Inner Shareable	Normal Non-cacheable	Normal Inner Non-cacheable Outer Cacheable

Continued on next page

Table C2.5 – Continued from previous page

Armv8 type	Armv8 shareability	LTI attribute when LTI_OUTER_CACHEABLE_MODE is	
		True	False
Normal-iNC-oWB	Non-shareable, Outer Shareable, Inner Shareable	Either: – Non-snoopable Write-Back No-allocate – Non-snoopable Write-Back Allocate Depending on the transaction type and Armv8 Outer Read-Allocate and Write-Allocate hints, as described in Table C2.6	Normal Inner Non-cacheable Outer Cacheable
Normal-iWB-oWB	Outer Shareable, Inner Shareable	Either: – Snoopable Write-Back No-allocate – Snoopable Write-Back Allocate Depending on the transaction type and Armv8 Outer Read-Allocate and Write-Allocate hints, as described in Table C2.6	Either: – Snoopable Write-Back No-allocate – Snoopable Write-Back Allocate Depending on the transaction type and Armv8 Outer Read-Allocate and Write-Allocate hints, as described in Table C2.6
Normal-iWB-oWB	Non-shareable	Either: – Snoopable Write-Back No-allocate – Snoopable Write-Back Allocate Depending on the transaction type and Armv8 Outer Read-Allocate and Write-Allocate hints, as described in Table C2.6	Either: – Non-snoopable Write-Back No-allocate – Non-snoopable Write-Back Allocate Depending on the transaction type and Armv8 Outer Read-Allocate and Write-Allocate hints, as described in Table C2.6

For transactions with Armv8 memory type Normal-iWB-oWB, [Table C2.6](#) shows how the choice between No-allocate and Allocate in LTI is chosen.

Table C2.6: Armv8 to LTI allocation hint mapping

LATRANS	LTI allocation hint depends on
R, R-CMO, R-DCMO	Armv8 Outer Read-Allocate
W, W-DCP, RW, W-CMO, DCP	Armv8 Outer Write-Allocate
CMO, DCMO, SPEC, DHCMO	Always allocate

Although this section defines an allocation hint for all transaction types, it is ignored by the system for many transactions. For example:

- SPEC transactions are terminated after translation is complete, and so the allocation hint is ignored.
- CMO, R-CMO, DCMO, R-DCMO, DHCMO, and W-CMO transactions are designed to de-allocate memory locations from caches and so the allocation hint is expected to be ignored by the system.

- DCP and W-DCP transactions are explicit cache allocating transactions, so the allocation hint is expected to be ignored by the system.

C2.4 DTI_TBU_TRANS_FAULT.FAULT_TYPE mapping

Table C2.7 shows the mapping for DTI_TBU_TRANS_FAULT.FAULT_TYPE.

Table C2.7: DTI_TBU_TRANS_FAULT.FAULT_TYPE mapping

DTI_TBU_TRANS_FAULT.FAULT_TYPE	LRRESP
NonAbort	TerminateRAZWI
Abort	FaultAbort. This value of DTI_TBU_TRANS_FAULT.FAULT_TYPE does not occur when LATRANS = SPEC, DCP, or DHCMO.
StreamDisabled	If (LATRANS = SPEC, DCP, or DHCMO) TerminateRAZWI, else FaultAbort.
GlobalDisabled	If (LATRANS = SPEC, DCP, or DHCMO) TerminateRAZWI, else FaultAbort.
TranslationPRI	FaultPRI This value of DTI_TBU_TRANS_FAULT.FAULT_TYPE does not occur when LATRANS = SPEC, DCP, or DHCMO.
TranslationStall	Not applicable

Chapter C3

Considerations for PCIe

This appendix describes the integration for PCIe. It contains the following section:

- [C3.1 PCIe integration](#)

C3.1 PCIe integration

This section describes recommendations for PCIe integration.

The LTI_MMU property must be True for PCIe Root Ports.

[Table C3.1](#), [Table C3.4](#), and [Table C3.5](#) describe how a PCIe RP should drive certain LA channel signals.

Table C3.1: Request channel PCIe integration

Signal	Description
LAVC	A PCIe RP implementation is permitted, but not required, to use multiple LTI Virtual Channels to help break protocol dependencies. See B2.2 Virtual Channels. Root Ports implementing LTI interfaces can be one of two types: 1. Fully Buffered. A Fully Buffered implementation can temporarily backpressure for flow control reasons but must not require progress of downstream transactions before it can accept responses on the LR channel. This requires implementing a buffer large enough to accept all outstanding LR responses. 2. Backpressuring. Backpressure implementation can stop granting credits on the LR channel when the RP cannot issue downstream transactions into the system. As a result, LR responses are prevented from being accepted for LA requests that have already been issued. It is recommended that the LTI request is performed before the point of ordering in the PCIe RP, typically using Fully Buffered implementation. For Fully Buffered LTI implementations, posted and non-posted requests can use the same VC. For Backpressuring LTI implementations, different Virtual Channels are used for posted and non-posted requests to ensure that posted requests can forward progress when non-posted requests cannot. Different Virtual Channels can also be used for different traffic classes.
LAOGV	For Fully Buffered implementations this signal is 0, because they can accept LR channel responses in any order. Backpressuring implementations must use order groups to ensure that posted writes remain in order where required.
LAFLOW	If the request is an ATS translated request, this signal is 1, ATST. Otherwise this signal is 2, NoStall. LAFLOW must not be PRI or STALL.
LAMMUV	This signal is 1.
LASID	LASID[15:0] is the Requester ID, otherwise known as BDF (Bus, Device, Function). Higher-order bits of LASID uniquely identify the PCIe segment in the StreamID space that is used by the SMMU.
LAIDENT	This signal is recommended to be 0.

[Table C3.3](#) shows the Non-Flit Mode mapping for **LASSIDV**, **LASSID**, **LAPRIV**, **LAINST**, and **LAATTR** signals.

Table C3.2: Request channel PCIe integration for Non-Flit Mode

Signal	Description
LASSIDV	If the request has a PASID header, this signal is 1. Otherwise, the signal is 0.
LASSID	This signal is the PASID.
LAPRIV	If the request has a PASID header and Priv = 1, this bit is 1. Otherwise, this bit is 0.
LAINST	If the request has a PASID header and Exe = 1, this bit is 1. Otherwise, this bit is 0.
LAATTR	The following encodings apply: If No_snoop is set If No_snoop is not set and TH is 0 If No_snoop is not set and TH is 1 Normal Non-cacheable Snoopable Write-Back No-allocate Snoopable Write-Back Allocate

Table C3.3 shows the Flit Mode for **LASSIDV**, **LASSID**, **LAPRIV**, **LAINST**, and **LAATTR** signals.

Table C3.3: Request channel PCIe integration for Flit Mode

Signal	Description
LASSIDV	This is the PV bit if OHC-A1 is present and PV is 1. Otherwise, this is 0.
LASSID	This is the PASID field if OHC-A1 is present and PV is 1. Otherwise, this is 0.
LAPRIV	This is the PMR bit if OHC-A1 is present and PV is 1. Otherwise, this is 0.
LAINST	This is the ER bit if OHC-A1 is present and PV is 1. Otherwise, this is 0.
LAATTR	The following encodings apply: If No_snoop is set If No_snoop is not set, and either OHC-B is not present or HV[0] is 0 If No_snoop is not set, OHC-B is present, and HV[0] is 1 Non-cacheable Snoopable Write-Back No-allocate Snoopable Write-Back Allocate

When the XT Mode is enabled, Table C3.4 shows mapping from the PCIe T and XT bits to **LASECSID**, **LAPASUNKNOWN**, and **LAPAS**.

Table C3.4: Mapping PCIe T and XT to LA

XT	T	LASECSID	LAPASUNKNOWN	LAPAS
0	0	Non-secure	0	Non-secure
0	1	Realm	1	Realm
1	0	Realm	0	Non-secure
1	1	Realm	0	Realm

When the XT mode is not supported or enabled, [Table C3.5](#) shows the mapping from the PCIe T bit to **LASECSID** and **LAPAS**.

Table C3.5: Mapping PCIe T to LA

T	LASECSID	LAPASUNKNOWN	LAPAS
0	Non-secure	0	Non-secure
1	Realm	0	Realm

The Secure and Root worlds are not intended to be accessed by PCIe Root Ports.

Chapter C4

Considerations for coherent interfaces

This appendix describes using LTI for transactions originated from devices with coherent interfaces, such as CXL.cache or CHI. The requests from CXL.cache are converted to the CHI protocol before they use LTI for translation. The coherent interfaces are expected to use translated addresses to enable the coherency protocol. The snoop request does not come to LTI and is not covered in this specification. The transaction request comes to LTI and does not require stage 1 or stage 2 translation. The SMMU only performs permission checks if it is configured in such a way, including *Granule Protection Checks* (GPC) and *Device Permission Table* (DPT) checks. It contains the following sections:

- [C4.1 LTI properties](#)
- [C4.2 LA mapping from CHI](#)
- [C4.3 LATRANS mapping](#)
- [C4.4 LAATTR mapping](#)
- [C4.5 LRATTR mapping](#)
- [C4.6 LRMECID mapping](#)
- [C4.7 LRHWATTR mapping](#)
- [C4.8 CHI opcode considerations](#)
- [C4.9 CHI completion response flit opcode mapping](#)
- [C4.10 Transaction flow examples](#)

C4.1 LTI properties

[Table C4.1](#) shows the recommended LTI property values when mapping to CHI properties.

Table C4.1: LTI properties

LTI property	Value
LTI_SSID_WIDTH	0
LTI_OG_WIDTH	0
LTI_LRADDR_WIDTH	Corresponds to the CHI property, Req_Addr_Width.
LTI_TLBLOC_WIDTH	0
LTI_GPC	True.
LTI_MMU	True.
LTI_PASUNKNOWN_PRESENT	False

C4.2 LA mapping from CHI

When mapping from CHI, the **LAMMUV** signal is driven depending on whether an SMMU translation is required. If the CHI request has come from CXL.cache, **LAMMUV** must be 0.

[Table C4.2](#) shows the recommended mapping of the CHI request flit to LA channel signals.

Table C4.2: LA mapping

LA signal	When LAMMUV is	
	0	1
LAFLOW	Not valid.	Set to ATST.
LASECSID	Not valid.	LASECSID = SecSID1 ? Realm : Non-secure
LASID	Not valid.	LASID [15:0] is the CHI request flit, StreamID. Higher-order bits of LASID uniquely identify the host port Segment Number in the StreamID space that is used by the SMMU.
LASSIDV	Not valid.	Set to 0.
LAOGV	It is recommended for the Host Port to maintain the order and set LAOGV to 0. However, if the Host Port expects the LTI responses to be in order, this bit can be driven to 1 to create a single order group for such transactions.	
LAADDR	LAADDR [LTI_LRADDR_WIDTH-1:0] is the CHI request flit, Addr. LADDR [63:LTI_LRADDR_WIDTH] is 0.	
LAPAS	This signal is CHI request flit, PAS.	
LAIDENT	Not valid.	Set to 1.
LAPM	Not valid.	Set to 0.
LAPRIV	Not valid.	Set to 0.
LAINST	Not valid.	Set to 0.

C4.3 LATRANS mapping

Table C4.3 shows the recommended mappings of the CHI request flit, Opcode, to the **LATRANS** values for IO coherent transactions.

Table C4.3: LATRANS mapping for IO coherent transactions

CHI request flit, Opcode	LATRANS
ReadNoSnP, ReadOnce	R
ReadOnceCleanInvalid	R-CMO
ReadOnceMakeInvalid	R-DCMO
WriteNoSnP, WriteNoSnPDef, WriteNoSnPZero, WriteUniquePtl, WriteUniqueFull, WriteUniqueZero, WriteUniquePtlStash and WriteUniqueFullStash with StashNIDValid = 0 and StashLPIDValid = 0	W
WriteNoSnPPtl+(P)CMO, WriteNoSnPFull+(P)CMO, WriteUniquePtl+(P)CMO, WriteUniqueFull+(P)CMO	W-CMO
WriteUniquePtlStash and WriteUniqueFullStash with StashNIDValid = 1 or StashLPIDValid = 1	W-DCP
StashOnce*	DCP
CleanShared, CleanSharedPersist*, CleanInvalid, CleanInvalidPoPA, CleanInvalidStorage	CMO
MakeInvalid	DCMO
AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare	RW

Table C4.4 shows the recommended mappings of the CHI request flit, Opcode, to the **LATRANS** values for fully coherent transactions.

When DPT is enabled, the write access check in DPT is not enforced for fully coherent transactions in this issue of the specification.

Table C4.4: LATRANS mapping for fully coherent transactions

CHI request flit, Opcode	LATRANS
ReadShared	R
ReadClean	
ReadNotSharedDirty	
ReadUnique	
ReadPreferUnique	
MakeReadUnique	
MakeUnique	
CleanUnique	
Evict	
WriteEvictFull	
WriteEvictOrEvict	
WriteCleanFull	
WriteCleanFull+(P)CMO	
WriteBack	
WriteBackFull+(P)CMO	

The CHI protocol transaction, PrefetchTgt, is not supported for mappings to LTI.

If a PrefetchTgt transaction is received, it is recommended that the request is discarded.

When WriteUniquePtlStash or WriteUniqueFullStash transaction has **LRRESP** = Downgrade1, output StashNIDValid = 0 and StashLPIDValid = 0.

C4.4 LAATTR mapping

In a coherent memory system, all coherent Managers must use consistent attributes to the same memory location. Therefore, the mapping of Armv8 memory types to AMBA[®] memory types must be consistent.

Table C4.5 shows the recommended mappings of the CHI request fields, MemAttr, SnpAttr, and Order to LAATTR values.

Table C4.5: LAATTR mapping

CHI request flit fields						CHI memory type	LAATTR
MemAttr[3:0]				SnpAttr	Order[1:0]		
[1]	[3]	[2]	[0]				
Device	Allocate	Cacheable	EWA				
1	0	0	0	0	11	Device-nRnE	Device-nGnRnE
	0	0	1	0	11	Device-nRE	Device-nGnRE
	0	0	1	0	00, 10	Device-RE	Device-nGRE
0	0	0	0	0	00, 10	Non-cacheable Non-bufferable	Non-cacheable
	0	0	1	0	00, 10	Non-cacheable Bufferable	
	0	1	1	0	00, 10	Non-snoopable Write-Back No-allocate	Normal Write-Back No-allocate
	1	1	1	0	00, 10	Non-snoopable Write-Back Allocate	Normal Write-Back Allocate
	0	1	1	1	00, 10	Snoopable Write-Back No-allocate	Snoopable Write-Back No-allocate
	1	1	1	1	00, 10	Snoopable Write-Back Allocate	Snoopable Write-Back Allocate

C4.5 LRATTR mapping

In a coherent memory system, all coherent Managers must use consistent attributes to the same memory location. Therefore, the mapping of Armv8 memory types to AMBA[®] memory types must be consistent.

Table C4.6 shows the recommended mappings of the CHI request flit fields, MemAttr, SnpAttr, and Order to **LRATTR** values.

Table C4.6: LRATTR mapping

LRATTR	CHI request flit fields						CHI memory type
	MemAttr[3:0]				Order[1:0]		
	[1] Device	[3] Allocate	[2] Cacheable	[0] EWA			
Device-nGnRnE	1	0	0	0	0	11	Device-nRnE
Device-nGnRE		0	0	1	0	11	Device-nRE
Device-nGRE Device-GRE		0	0	1	0	00, 10	Device-RE
Normal Non-cacheable, Normal Inner Non-cacheable Outer Cacheable ^a	0	0	0	0	0	00, 10	Non-cacheable Bufferable
Non-snoopable Write-Back No-allocate		0	1	1	0	00, 10	Non-snoopable Write-Back No-allocate
Non-snoopable Write-Back Allocate		1	1	1	0	00, 10	Non-snoopable Write-Back Allocate
Snoopable Write-Back No-allocate		0	1	1	1	00, 10	Snoopable Write-Back No-allocate
Snoopable Write-Back Allocate		1	1	1	1	00, 10	Snoopable Write-Back Allocate

^a Normal Inner Non-cacheable Outer Cacheable is only possible when LTI_OUTER_CACHEABLE_MODE is False.

In instances where WriteEvictFull and WriteEvictOrEvict transactions require MemAttr to be Allocate, the LTI Manager is recommended to ignore the allocation hint in **LRATTR** and output MemAttr as Allocate.

In instances where the ReadOnceMakeInvalid transaction requires MemAttr to be No-allocate, the LTI Manager is recommended to ignore the allocation hint in **LRATTR** and output MemAttr as No-allocate.

When **LATRANS** != CMO or DCMO, the CHI transaction issued downstream is recommended to retain the incoming SnpAttr and MemAttr attributes for Device and Cacheable. The MemAttr attributes for Allocate and EWA can be modified by the translation.

When a WriteUniquePtlStash or WriteUniqueFullStash transaction has **LRRESP** = Downgrade1, the output is StashNIDValid = 0 and StashLPIDValid = 0.

If opcode is WriteNoSnpDef, the CHI memory must not be Normal WriteBack. If the final CHI memory type is Normal WriteBack, it is recommended that the transaction is terminated with a NDERR response.

If the opcode is WriteNoSnpPtlCleanInv, WriteNoSnpPtlCleanInvPoPA, WriteNoSnpFullCleanInv or WriteNoSnpFullCleanInvPoPA, and the final CHI memory type is Snoopable WriteBack, it is recommended that the transaction is terminated with a NDERR response.

C4.6 LRMECID mapping

It is recommended to map **LRMECID** as follows:

- When **LAMMUV** is 1 and **LRPAS** is not NSP, the output MECID in the request flit is **LRMECID**.
- When **LAMMUV** is 0 or **LRPAS** is NSP, the output MECID in the request flit is the supplied MECID or 0 if there is no supplied MECID. It is IMPLEMENTATION DEFINED how the MECID is supplied.

C4.7 LRHWATTR mapping

It is recommended to map **LRHWATTR** as follows:

- When **LAMMUV** is 1, the output PBHA in the request flit is **LRHWATTR**.
- When **LAMMUV** is 0, the output PBHA in the request flit is the supplied PBHA or 0 if there is no supplied PBHA. It is IMPLEMENTATION DEFINED how the PBHA is supplied.

C4.8 CHI opcode considerations

There are some CHI opcodes that can not use the Normal WriteBack memory type. When using FWB, these must be modified or terminated.

It is recommended to modify the transaction opcode as in [Table C4.7](#). The modification can happen if **LRRESP** is a downgrade type that changes the opcode, or the translated attributes are not supported by the opcode.

Table C4.7: CHI opcode modifications

Input opcode	Output opcode	Conditions
ReadNoSnp	ReadOnce	Final memory type is Snoopable Write-Back
WriteNoSnpPtl	WriteUniquePtl	Final memory type is Snoopable Write-Back
WriteNoSnpFull	WriteUniqueFull	Final memory type is Snoopable Write-Back
WriteNoSnpZero	WriteUniqueZero	Final memory type is Snoopable Write-Back
MakeInvalid	CleanInvalid	LRRESP is Downgrade2
ReadOnceMakeInvalid	ReadOnceCleanInvalid	LRRESP is Downgrade2
WriteNoSnpPtlCleanSh	WriteUniquePtlCleanSh	Final memory type is Snoopable Write-Back
WriteNoSnpPtlCleanShPerSep	WriteUniquePtlCleanShPerSep	Final memory type is Snoopable Write-Back
WriteNoSnpFullCleanSh	WriteUniqueFullCleanSh	Final memory type is Snoopable Write-Back
WriteNoSnpFullCleanShPerSep	WriteUniqueFullCleanShPerSep	Final memory type is Snoopable Write-Back
WriteNoSnpFullCleanInvStrg	WriteUniqueFullCleanInvStrg	Final memory type is Snoopable Write-Back

Note

CHI transactions always have **LAFLOW** = ATST and **LAIDENT** = 1. So, a Snoopable transaction can always retain its Snoopable Write-Back attributes, and therefore it never needs to modify the opcode due to memory type mismatch. For example, a ReadOnce transaction never needs to be converted to ReadNoSnp.

C4.9 CHI completion response flit opcode mapping

[Table C4.8](#) shows the relative mapping of **LRRESP** to the CHI response flit fields, **RespErr** and **Resp** when the transaction is terminated by the LTI Manager resulting from a translation fault.

Table C4.8: CHI completion response flit opcode mapping

LRRESP	CHI completion flits, RespErr and Resp
Success Downgrade1 Downgrade2	RespErr, Resp from downstream
FaultAbort	NDERR
TerminateRAZWI	OK
FaultPRI	Not applicable

C4.10 Transaction flow examples

The examples described in this section include:

- [C4.10.1 Coherent read from CHI C2C device](#)
- [C4.10.2 Coherent write from CXL cache device](#)

The examples described in these subsections do not include interactions between the HN-F and Request Node or Subordinate Node.

The examples use PCIe ATS to translate the Virtual Address from the Device.

C4.10.1 Coherent read from CHI C2C device

Figure C4.1 shows an example of a ReadNotSharedDirty transaction for a memory access.

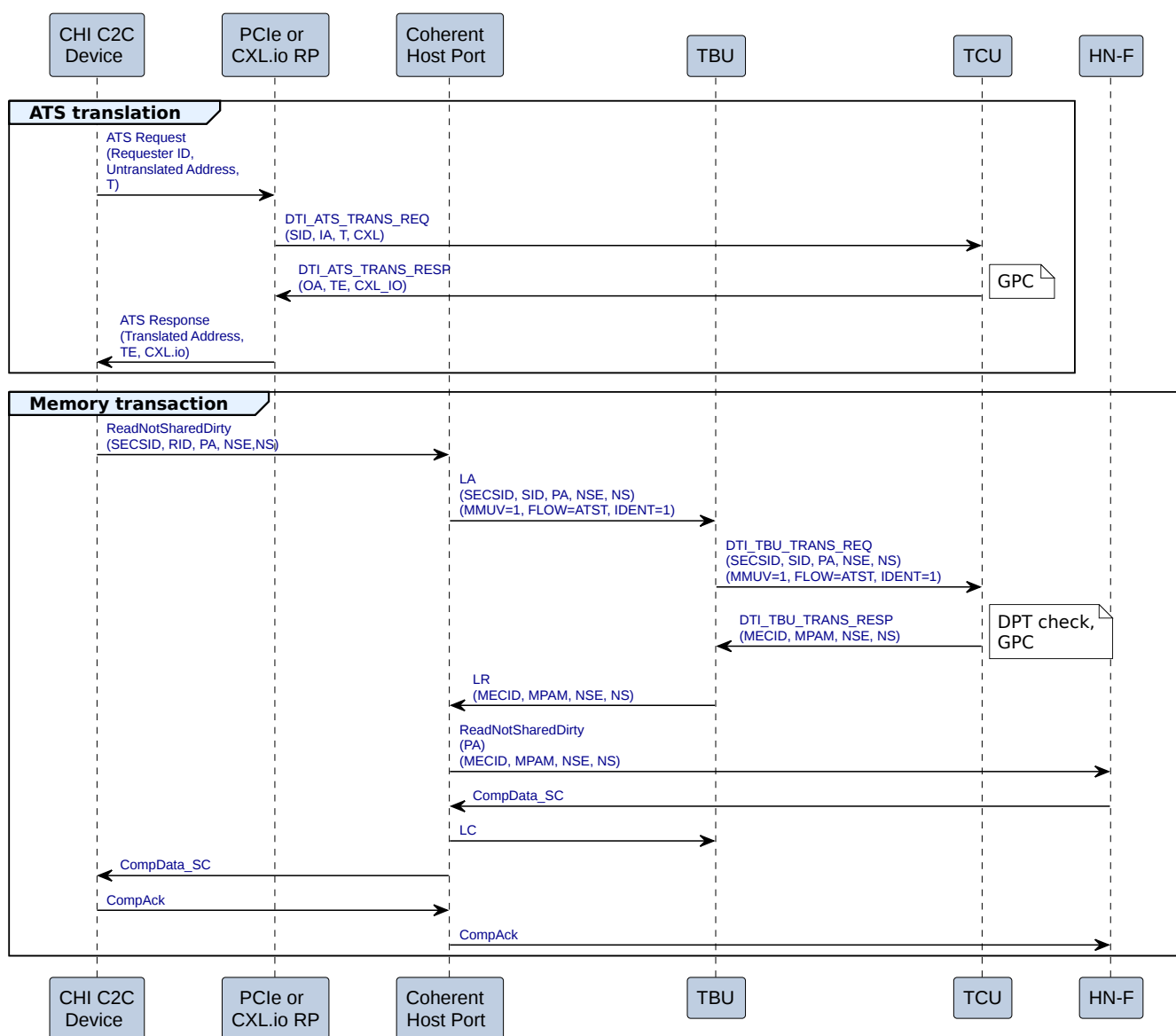


Figure C4.1: Example of a CHI Read transaction

The sequence for [Figure C4.1](#) includes:

1. The CHI C2C Device sends an ATS Request to the PCIe or CXL.io RP.
2. The PCIe or CXL.io RP sends a DTI-ATS translation request to the TCU.
If CXL is 1, the request originates from a coherent device.
3. The TCU performs the stage 1 and stage 2 translations and GPC.
4. The TCU sends the DTI-ATS translation response to the PCIe or CXL.io RP. The PCIe or CXL.io RP provides the translated PA, TO, and CXL_IO.
If CXL_IO is 0, the transaction is permitted to be presented as a CHI snoopable request.
5. The PCIe or CXL.io RP sends the ATS response to the CHI C2C Device.
6. The CHI C2C Device sends a ReadNotSharedDirty request to the Coherent Host Port.
7. The Coherent Host Port sends an LA message to the TBU with **LAMMUV** = 1, **LAFLOW** = ATST, and **LAIDENT** = 1.
8. The TBU converts the LA message into a DTI-TBU translation request and sends it to the TCU.
9. The TCU performs a GPC and DPT check.
If both checks pass, the TCU sends the DTI-TBU translation response to the TBU with the translated MECID, MPAM, and PAS information, where PAS is encoded as {NSE, NS}.
10. The TBU converts the DTI-TBU translation response to an LR message and sends it to the Coherent Host Port.
11. The Coherent Host Port sends a ReadNotSharedDirty request to the HN-F.
12. The HN-F resolves coherency and returns a CompData_SC response to the Coherent Host Port.
13. The Coherent Host Port sends an LC message to the TBU to complete the translation.
In parallel, the Coherent Host Port also sends a CompData_SC response to the CHI C2C Device.
14. The CHI C2C Device sends a CompAck response to the Coherent Host Port.
15. The Coherent Host Port sends a CompAck response to the HN-F.

C4.10.2 Coherent write from CXL.cache device

[Figure C4.2](#) shows an example of a D2H.DirtyEvict transaction for a memory access.

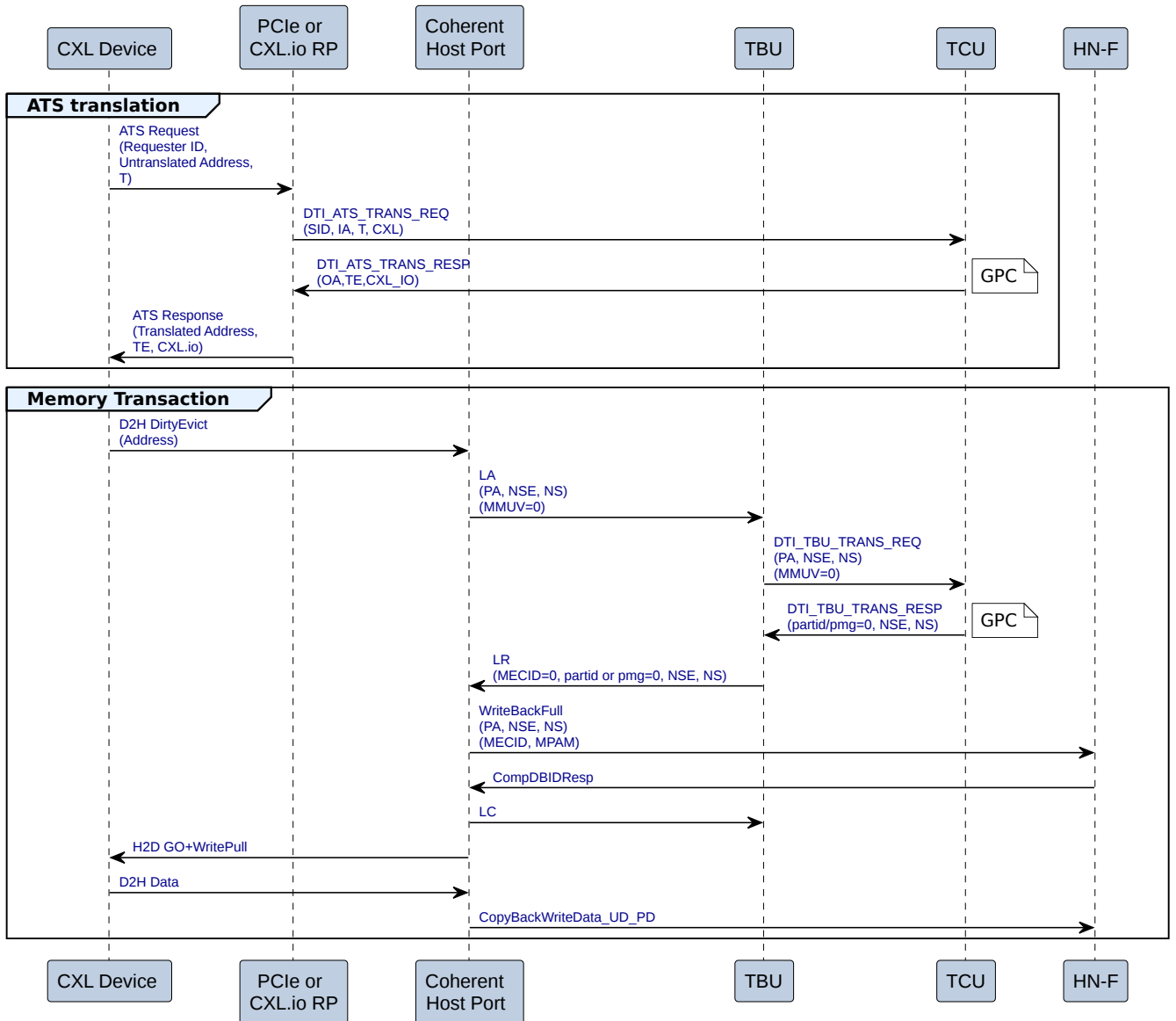


Figure C4.2: Example of a CXL write transaction

The sequence for Figure C4.2 includes:

1. The CXL Device sends an ATS Request to the PCIe or CXL.io RP.
2. The PCIe or CXL.io RP sends a DTI-ATS translation request to the TCU.
If CXL is 1, the request originates from a coherent device.
3. The TCU performs the stage 1 and stage 2 translations and GPC.
4. The TCU sends the DTI-ATS translation response to the PCIe or CXL.io RP. The PCIe or CXL.io RP provides the translated PA, TO, and CXL_IO.
If CXL_IO is 0, the transaction is permitted to be presented on a CXL.cache interface.
5. The PCIe or CXL.io RP sends the ATS response to the CXL Device.
6. The CXL Device sends a D2H DirtyEvict message to the Coherent Host Port.
7. The Coherent Host Port sends an LA message to the TBU with **LAMMUV** = 0.

8. The TBU converts the LA message into a DTI-TBU translation request and sends it to the TCU.
9. The TCU performs a GPC.
If the check passes, the TCU sends the DTI-TBU translation response to the TBU.
There is no valid MECID at this point.
MPAM PARTID and PMG are 0.
The translated PAS is equal to incoming PAS.
10. The TBU converts the DTI-TBU translation response to an LR message and sends it to the Coherent Host Port, with MECID = 0, MPAM PARTID = 0, and PMG = 0.
11. The Coherent Host Port sends a WriteBackFull request to the HN-F.
12. The HN-F resolves coherency and returns a CompDBIDResp response to the Coherent Host Port.
13. The Coherent Host Port sends an LC message to the TBU to complete the translation.
In parallel, the Coherent Host Port also sends a H2D combined GO_WritePull message to the CXL Device.
14. The CXL Device sends a D2H Data message to the Coherent Host Port.
15. The Coherent Host Port sends a CopyBackWriteData_UD_PD response to the HN-F.

Chapter C5

Signal list

This appendix describes which signals are present on LTI-A and LTI-B interfaces. It contains the following section:

- [C5.1 Signal list](#)

C5.1 Signal list

Table C5.1 shows of all the signals with codes that describe the presence requirements for each LTI version. The Presence column describes the property condition used to specify the presence of the signal.

The following key is used:

- Y** Signal is present.
- N** Signal is not present.
- O** Signal is optional.

Table C5.1: Summary of signal presence for each LTI version

Signal	Presence	LTI issue			
		A A.b	B	C	D
CLK	-	O	O	O	O
RESETn	-	O	O	O	O
LAVALID	-	Y	Y	Y	Y
LAVC	LTI_VC_COUNT > 1	O	O	O	O
LACREDIT	-	Y	Y	Y	Y
LAID	LTI_ID_WIDTH > 0	O	O	O	O
LAOGV	-	Y	Y	Y	Y
LAOG	LTI_OG_WIDTH > 0	O	O	O	O
LAFLOW	LTI_MMU	Y	O	O	O
LAMMUV	-	N	Y	Y	Y
LASECSID	LTI_MMU	Y	O	O	O
LASID	LTI_MMU and LTI_SID_WIDTH > 0	O	O	O	O
LASSIDV	LTI_MMU and LTI_SSID_WIDTH > 0	Y	O	O	O
LASSID	LTI_MMU and LTI_SSID_WIDTH > 0	Y	O	O	O
LAPM	LTI_MMU and LTI_GDI	N	N	N	O
LAPASUNKNOWN	LTI_PASUNKNOWN_PRESENT	N	N	N	O
LAPROT[2]^b	LTI_MMU	Y	O	O	N
LAPROT[1]^b	-	Y	Y	Y	N
LAPROT[0]^b	LTI_MMU	Y	O	O	N
LANSE^b	LTI_GPC	N	O	O	N
LAPAS	-	N	N	N	Y
LAPRIV	LTI_MMU	N	N	N	O
LAINST	LTI_MMU	N	N	N	O

Continued on next page

Table C5.1 – Continued from previous page

Signal	Presence	LTI issue			
		A A.b	B	C	D
LAADDR	-	Y	Y	Y	Y
LATRANS	-	Y	Y	Y	Y
LAATTR	-	Y	Y	Y	Y
LALOOP	LTI_LOOP_WIDTH > 0	O	O	O	O
LATLBLOC	LTI_TLBLOC_WIDTH > 0	O	O	O	O
LAHWATTR^a	LTI_LAHWATTR_PRESENT ^a	N	O	N	N
LAMECID^a	LTI_MECID_WIDTH > 0	N	O	N	N
LAUSER	LTI_LAUSER_WIDTH > 0	O	O	O	O
LAIDENT	LTI_MMU	N	O	O	O
LRVALID	-	Y	Y	Y	Y
LRVC	LTI_VC_COUNT > 1	O	O	O	O
LRCREDIT	-	Y	Y	Y	Y
LRID	LTI_ID_WIDTH > 0	O	O	O	O
LRCTAG	-	Y	Y	Y	Y
LRRESP	-	Y	Y	Y	Y
LRPROT[2]^b	LTI_MMU	Y	O	O	N
LRPROT[1]^b	-	Y	Y	Y	N
LRPROT[0]^b	LTI_MMU	Y	O	O	N
LRNSE^b	LTI_GPC	N	O	O	N
LRPAS	-	N	N	N	Y
LRADDR	-	Y	Y	Y	Y
LRATTR	-	Y	Y	Y	Y
LRHWATTR	-	Y	Y	Y	Y
LRMPAM	LTI_MPAM_SUPPORT != False	Y	Y	O	O
LRMECID	LTI_MECID_WIDTH > 0	N	O	O	O
LRSIZE	LTI_CACHE_SUPPORT	N	N	N	Y
LRLOOP	LTI_LOOP_WIDTH > 0	O	O	O	O
LRUSER	LTI_LRUSER_WIDTH > 0	O	O	O	O
LCVALID	-	Y	Y	Y	Y
LCCREDIT	-	Y	Y	Y	Y
LCCTAG	-	Y	Y	Y	Y

Continued on next page

Table C5.1 – Continued from previous page

Signal	Presence	LTI issue			
		A A.b	B	C	D
LCUSER	LTI_LCUSER_WIDTH > 0	O	O	O	O
LTVALID	LTI_CACHE_SUPPORT	N	N	N	O
LTCREDIT	LTI_CACHE_SUPPORT	N	N	N	O
LTCTAG	LTI_CACHE_SUPPORT	N	N	N	O
LTUSER	LTI_CACHE_SUPPORT and LTI_LTUSER_WIDTH > 0	N	N	N	O
LMOPENREQ	-	Y	Y	Y	Y
LMOPENACK	-	Y	Y	Y	Y
LMACTIVE	-	Y	Y	Y	Y
LMASKCLOSE	-	Y	Y	Y	Y

^a The **LAMECID** and **LAHWATTR** signals and the LTI_LAHWATTR_PRESENT property are only defined on an LTI-B interface. The **LAMECID** and **LAHWATTR** signals are not described in this issue of the specification. For more details, see Issue B.

^b The **LAPROT**, **LANSE**, **LRPROT**, and **LRNSE** signals are only defined on LTI-C and earlier interfaces. These signals are not described in this issue of the specification. For more details, see Issue C.

Chapter C6

Parity check signal matrix

Parity check signals are not present when Check_Type property is False.

Parity check signals for each interface type are shown in [Table C6.1](#), using the codes defined in [Table C5.1](#).

Table C6.1: Summary of parity signal presence of each LTI version

Name	LTI-A LTI-B LTI-C	LTI-D
LAVALIDCHK	N	O
LAVCCHK	N	O
LACREDITCHK	N	O
LAIDCHK	N	O
LAOGCHK	N	O
LASECSIDCHK	N	O
LASIDCHK	N	O
LASSIDCHK	N	O
LACTLCHK0	N	O
LACTLCHK1	N	O

Continued on next page

Table C6.1 – Continued from previous page

Name	LTI-A LTI-B LTI-C	LTI-D
LACTLCHK2	N	O
LAPMCHK	N	O
LAPASUN- KNOWNCHK	N	O
LAPASCHK	N	O
LAADDRCHK	N	O
LALOOPCHK	N	O
LATLBLOCCHK	N	O
LAUSERCHK	N	O
LRVALIDCHK	N	O
LRVCCHK	N	O
LRCREDITCHK	N	O
LRIDCHK	N	O
LRCTAGCHK	N	O
LRRESPCHK	N	O
LRPASCHK	N	O
LRADDRCHK	N	O
LRCTLCHK0	N	O
LRMPAMCHK	N	O
LRMECIDCHK	N	O
LRSIZECHK	N	O
LRLOOPCHK	N	O
LRUSERCHK	N	O
LCVALIDCHK	N	O
LCCREDITCHK	N	O
LCCTAGCHK	N	O
LCUSERCHK	N	O
LTVALIDCHK	N	O
LTCREDITCHK	N	O
LTCTAGCHK	N	O
LTUSERCHK	N	O
LMOPENREQCHK	N	O

Continued on next page

Table C6.1 – Continued from previous page

Name	LTI-A LTI-B LTI-C	LTI-D
LMOPENACKCHK	N	O
LMACTIVECHK	N	O
LMASKCLOSECHK	N	O

Chapter C7

Revisions

This appendix describes the technical changes between released issues of this specification. It contains the following sections:

- *C7.1 Changes between Issue A and Issue A.b*
- *C7.2 Changes between Issue A.b and Issue B*
- *C7.3 Changes between Issue B and Issue C*
- *C7.4 Changes between Issue C and Issue D*

C7.1 Changes between Issue A and Issue A.b

Table C7.1: Changes between Issue A and Issue A.b

Change	Location
Terminology update	Throughout the specification

C7.2 Changes between Issue A.b and Issue B

Table C7.2: Changes between Issue A.b and Issue B

Change	Location
Support for Realm Management Extension (RME): – Root and Realm address spaces – Granule Protection Check (GPC)	Table B3.1 Chapter B4 <i>Request channel</i> Chapter B5 <i>Response channel</i>
Support for Memory Encryption Contexts (MEC)	Table B3.1 Chapter B4 <i>Request channel</i> Chapter B5 <i>Response channel</i>
Support for AXI Issue J features: – UnstashTranslation – InvalidateHint – PBHA	Chapter B4 <i>Request channel</i> Chapter C1 <i>Considerations for AXI5</i>
Support for PASID header on ATS Translated transactions	Chapter B4 <i>Request channel</i> Chapter C3 <i>Considerations for PCIe</i>
Support for identity translation indication	Chapter B4 <i>Request channel</i> Chapter C3 <i>Considerations for PCIe</i>
Clarification of Virtual Channels	B2.2 <i>Virtual Channels</i> Table B3.1

C7.3 Changes between Issue B and Issue C

Table C7.3: Changes between Issue B and Issue C

Change	Location
Support for coherent interfaces	Chapter C4 Considerations for coherent interfaces
Relaxation of W-CMO	B4.3 Transaction attributes B5.3 Attribute restrictions for specific transaction types C1.4 LRATTR mapping
LRMPAM is extended for more Partition ID values	Table B5.1
New property, LTI_MPAM_SUPPORT	Table B3.1
Removal of LAHWATTR and LAMECID signals, including removal of LTI_LAHWATTR_PRESENT property	Throughout the specification
Memory attributes tightened for an ATST transaction with LAIDENT = 1	Table B5.1
Signal list is updated	Table C5.1

C7.4 Changes between Issue C and Issue D

Table C7.4: Changes between Issue C and Issue D

Change	Location
Translation cache support added to reduce request rate	B1.5 Translation cache Table B2.1 Table B3.1 Table B5.1 B12.5 LTI_CACHE_SUPPORT compatibility B6.2 Completion channel characteristics Chapter B7 Tag channel Chapter C2 Considerations for DTI
Width of CTAG signals extended to improve invalidation performance	B2.1 Transaction flow Table B3.1 Table B5.1 Table B6.1
Support for SMMUv3 GDI and PCIe XT mode added LAPROT and LANSE signals removed and replaced by LAPAS , LAPRIV , and LAINST signals LRPROT and LRNSE signals removed and replaced by LRPAS signal	B3.1 LTI properties Table B4.1 B4.5 Context and PAS combinations Table B5.1 B12.6 LTI_GDI compatibility B12.9 PAS signaling compatibility C3.1 PCIe integration Chapter C4 Considerations for coherent interfaces
Response channel does not signal Privileged/Unprivileged and Instruction/Data, where translated transactions are always Privileged/Data	Table B4.1 B5.1 Signals C1.9 PRIV and INST mapping B12.10 PRIV and INST signaling compatibility
Outer cacheable mode supported	B3.1 LTI properties B12.8 LTI_OUTER_CACHEABLE_MODE compatibility B4.3 Transaction attributes Table B4.4 Table B5.5 Table B5.1 C2.3 Attribute mapping C1.2 LAATTR mapping C1.4 LRATTR mapping Table C4.5 Table C4.6 Table C3.1 C4.8 CHI opcode considerations
W-DCP transactions are permitted to progress downstream without specifying a target cache if DCP permission is not granted	Table B4.2 Table B5.3 C1.4 LRATTR mapping Table C1.1

Continued on next page

Table C7.4 – Continued from previous page

Change	Location
Special transaction handling updated	C2.2 Special handling for specific LATRANS values
Maximum value for LTI properties	Table B3.1
CMO to PoPS	B5.3 Attribute restrictions for specific transaction types C1.1 LATRANS mapping C4.3 LATRANS mapping
Interface protection added	Chapter B11 Interface protection
Updated pseudocode	B13.1 PermissionCheckDHCMO
Updated signal matrix	C5.1 Signal list
New appendix for parity signal matrix	Chapter C6 Parity check signal matrix